14 haftalık yol haritasında 10. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 10

Karma (Hashing)

Karma Tabloları ve Desen Eşleme

MODÜL III · DİNAMİK VERİ VE HIZLI ERİŞİM · H8–H10

> **HAFTANIN BÜYÜK SORUSU**
>
> *Milyonlarca kayıt arasında bir anahtarı aramadan, neredeyse tek adımda nasıl bulursunuz ve bir saldırgan bu "sihri" nasıl bozabilir?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları.....4
1.1 Öğrenme çıktıları.....4
2. Geçen Haftadan ve AP-1'den Bağlantı.....5
3. Kavramsal Anlatım: Anahtardan İndise.....5
3.1 Sözlük soyut veri tipi.....5
3.2 Doğrudan adresleme ve sınırları.....6
3.3 Karma fonksiyonları ve düzgün dağılım.....7
3.4 Çakışmanın kaçınılmazlığı: güvercin yuvası ve doğum günü.....9
3.5 Kaputun Altında: maske, düşük bitler, önbellek ve işaretsiz taşma.....10
4. Algoritmalar.....11
4.1 Karma tablosu ve ayrı zincirleme.....11
4.2 Açık adresleme: doğrusal yoklama ve mezar taşı.....14
4.3 Yük faktörü ve yeniden boyutlandırma.....18
4.4 Naif desen eşleme.....20
4.5 Rabin–Karp: kayan karma.....22
4.6 KMP: önek fonksiyonu (merak).....25
5. Çözümlü Örnekler ve Uygulamalar.....26
5.1 Kelime frekansı (Örnek 7).....26
5.2 Yinelenen tespiti ve iki toplam (Örnek 8).....27
5.3 Anagram gruplama: anahtar tasarımı (Örnek 9).....29
5.4 Doğum günü simülasyonu (Örnek 10).....29
5.5 Uygulama kalıpları ve önbellekleme (H13'e köprü).....30
5.6 Güvenlik: karma taşkını ve savunmalar (Örnek 13).....30
6. Haftanın Deneyi: Yük Faktörü, Bellek Erişimi ve Saldırı.....32
6.1 Ölçüm düzeni.....32
6.2 Yük faktörü → ortalama yoklama: kuram ve ölçüm.....32
6.3 Aynı tablolarda arama süresi: bitişik hücreler ve dağınık düğümler.....33
6.4 Saldırı altında ekleme: karesel büyüme ve savunmalar.....33
6.5 Kelime sayacı: elle yazılmış tablo ve kütüphane.....34
6.6 Sınırlılıklar ve kendi makinenizde.....35
7. Hata Avcısı.....35
8. Dil Penceresi: C · C++ · Python.....36
9. Mühendis Gözüyle: Ortalama Durum Bir Varsayımdır.....38
10. Mini-OBS 2 v2.9: Numara Dizini, O(n) Yinelenen Tespiti ve Alt Dizgi Araması.....38

10.1 Gereksinimler 39
10.2 İpuçları 39
10.3 Başlangıç noktası 39
10.4 Kabul ölçütleri ve test senaryoları 40
10.5 Üçlü karşılaştırma: v2.1 → v2.5 → v2.9 41
11. Alıştırmalar ve Atölyeler 41
11.1 İzleme ve çıktı tahmini (A) 41
11.2 Karmaşıklık (B) 42
11.3 Değişmez yazma (C) 42
11.4 Hata bulma (D) 42
11.5 Parsons (E) 43
11.6 Kod yazma (F) 44
11.7 Diller arası okuma (G) 44
11.8 Atölye 1 · Karma fonksiyonu yarışması (eşli, ≈ 25 dk) 45
11.9 Atölye 2 · Kelime sayacı: üç dil, iki yöntem (gruplar, ≈ 20 dk) 45
11.10 Atölye 3 · Saldırı ve savunma (iki takım, ≈ 25 dk) 45
12. Kendini Değerlendirme 45
13. İleri Bakış ve Merak Kutusu 46
14. Özet, Kavram Haritası ve Terim Sözlüğü 46
15. Kaynaklar 47
Ek A. Dil Karşılıkları 49
A.1 Örnek 1 · Doğrudan adresleme 49
A.2 Örnek 2 · Dizgi karma fonksiyonları 50
A.3 Örnek 3 · Ayrı zincirleme 51
A.4 Örnek 4 · Doğrusal yoklama ve mezar taşı 52
A.5 Örnek 5 · Yük faktörü deneyi 53
A.6 Örnek 6 · Yeniden boyutlandırma 54
A.7 Örnek 7 · Kelime frekansı 55
A.8 Örnek 8 · Yinelenen tespiti ve iki toplam 56
A.9 Örnek 9 · Anagram gruplama 57
A.10 Örnek 10 · Doğum günü paradoksu 57
A.11 Örnek 11 · Naif desen eşleme 58
A.12 Örnek 12 · Rabin–Karp 59
A.13 Örnek 13 · Karma saldırısı 60
A.14 Örnek 14 · KMP önek fonksiyonu 61

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Milyonlarca kayıt arasında bir anahtarı aramadan, neredeyse tek adımda nasıl bulursunuz ve bir saldırgan bu "sihri" nasıl bozabilir?

H4'te sıralı bir dizide ikili aramanın $\lceil \log_2 n \rceil$ kadar inceleme yaptığını gördük: 10^6 kayıtta yaklaşık 20 inceleme. Bu hafta **karma tablosu** (hash table) ile aynı soruyu ortalama **bir iki** hücreye bakarak yanıtlayacağız. Mini-OBS 2 v2.9'un ölçümünde 10^6 kayıtta bir numarayı ikili aramayla bulmak 762 ns, karma tablosuyla 186 ns sürdü (4,1 kat; ortalama yoklama 1,000; v2.9 belgesi § 8.1). Aynı fikir H2'nin yinelenen numara tespitini de değiştirir: Örnek 8, $n = 10^5$ numarada yinelenenleri 107 133 hücre yoklamasıyla bulur; tüm çiftleri karşılaştıran kaba kuvvet $n(n-1)/2 = 4\,999\,950\,000$ karşılaştırma ister.

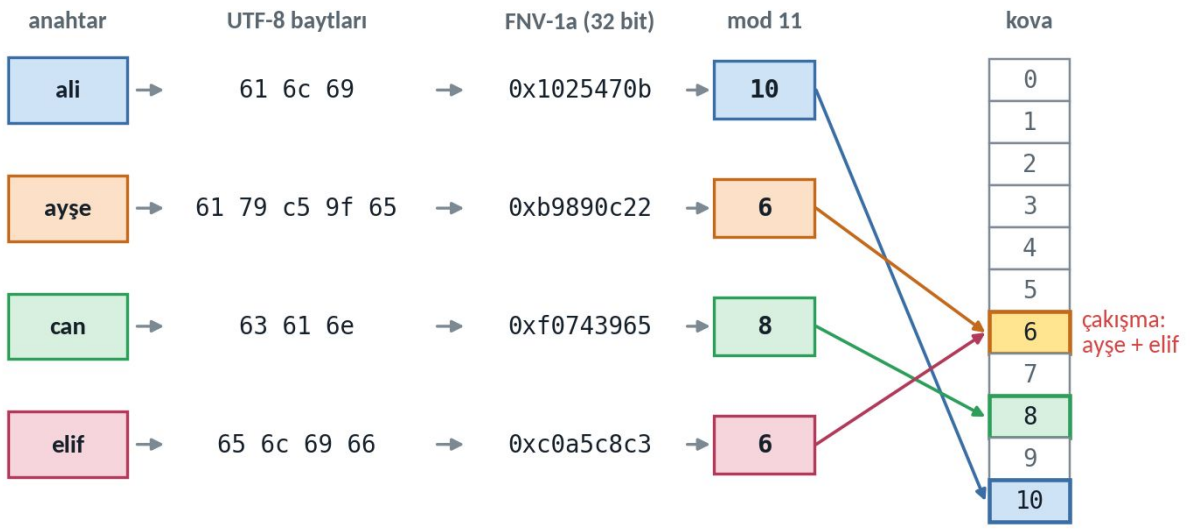
Sihrin kaynağı basittir: anahtar bir **karma fonksiyonuyla** (hash function) bir tamsayıya, o da bir dizi indisine çevrilir. İki soru doğar: farklı anahtarlar aynı indise düşerse (**çakışma**, collision) ne yaparız ve bedeli nedir? "Ortalama bir iki yoklama" anahtarların rastgele gibi dağıldığı varsayımına dayanır; anahtarları bir saldırgan seçerse ne olur? Haftanın ikinci yarısı aynı fikri **desen eşlemeye** (string matching) taşır: Rabin–Karp, metnin her penceresinin karmasını O(1)'de günceller.

> GERÇEK DÜNYADAN • **Ortalama O(1)'den O(n)'e: algoritmik karmaşıklık saldırıları**
>
> Crosby ve Wallach (2003) karma tablolarının ortalama durum varsayımını bir saldırı yüzeyi olarak ele aldı: hepsi aynı kovaya düşen anahtarlarla n ekleme $\Theta(n^2)$ iş olur. Perl 5.6.1 ve 5.8.0'da 90 000 kısa saldırı dizgisini eklemek hedef sürümde 6506 s ve 6838 s işlemci zamanı aldı (öbür sürümde 2 s'nin altında); Bro saldırı tespit sistemi saniyede 1200 yerine 24 paket işleyebildi ve çevirmeli modemden az bant genişliğiyle gönderilen paketler altı dakika sonra trafiğin %71'ine varan kısmını düşürttü.
>
> 28 Aralık 2011'de 28C3'te Alexander Klink ve Julian Wälde, "Effective Denial of Service attacks against web application platforms" konuşmasında popüler web dillerinin ve platformlarının çoğundaki ortak kusuru gösterdi (Klink & Wälde, 2011). Aynı gün yayımlanan oCERT-2011-003 Java, JRuby, PHP, Python, Rubinius ve Ruby'deki öngörülebilir çakışmaları bildirdi; sorun özellikle POST parametre sayısını sınırlamayan uygulamalarda ortaya çıkıyordu. Python'un 2.6.8, 2.7.3, 3.1.5 ve 3.2.3 düzeltmesi rastgeleleştirmeyi **kapalı** getirdi (`-R` ile açılıyordu; oCERT, 2011); 3.3 onu varsayılan yaptı (Python Software Foundation, t.y.-c), 3.4 `str`/`bytes` karmasını anahtarlı SipHash'e geçirdi (Heimes, 2013); bu makinedeki 3.11 siphash13 kullanır (Python Software Foundation, t.y.-b; Kavram 3).
>
> Bu hafta aynı saldırıyı kendi tablomuzda yeniden üretip hangi "savunmanın" gerçekten işe yaradığını ölçeceğiz (Örnek 13; Bölüm 5.6 ve 6.4).

1.1 Öğrenme çıktıları

- **ÖÇ 10.1** Doğrudan adresleme ile karma tablosunu karşılaştırır; iyi bir karma fonksiyonunun özelliklerini açıklar. (Bölüm 3.1–3.4, Örnek 1–2)
- **ÖÇ 10.2** Ayrı zincirleme ve doğrusal yoklamalı açık adreslemeyle karma tablosu (ekleme, arama, silme) gerçekler. (Bölüm 4.1–4.2, Örnek 3–4)
- **ÖÇ 10.3** Yük faktörünün başarım üzerindeki etkisini deneyle gösterir; yeniden boyutlandırmayı amortize analizle gerekçelendirir. (Bölüm 4.3, 6.2, Örnek 5–6)
- **ÖÇ 10.4** Karma tablolarını frekans sayma, yinelenen tespiti ve iki toplam gibi problemlerde kullanarak $O(n^2)$ çözümleri ortalama O(n)'e indirir. (Bölüm 5.1–5.3, 5.5, 10, Örnek 7–9)
- **ÖÇ 10.5** Naif desen eşleme ve Rabin–Karp (kayan karma) algoritmalarını gerçekler ve karmaşıklıklarını karşılaştırır. (Bölüm 4.4–4.6, Örnek 11–12, 14)

Şekil 10.1
Anahtar → karma → kova (Örnek 3): UTF-8 baytları, 32 bitlik FNV-1a, mod 11; ayşe ve elif aynı kovada (çakışma)



2. Geçen Haftadan ve AP-1'den Bağlantı

H8'in **bağlı listesi** ayrı zincirlemenin kovalarıdır ve `Dugum **` ile özel durumsuz silme (H8 Örnek 5) zincirden silmenin ta kendisidir; H8'in **dinamik dizisi** ve iki katına büyütmenin **amortize** analizi yeniden boyutlandırmada aynen kullanılır. H9'un **dairesel tamponu** `& (k - 1)` ile sarmanın yalnız k 2'nin kuvvetiyken `% k` ile aynı olduğunu göstermişti: kapasite seçimi (Bölüm 3.5). AP-1 H13'ten **karakter dizilerini** (UTF-8 baytları, `strcmp`, kopyalama), H1'in Dil Penceresi'nden **kalanın işaretini** kullanacağız.

> TAHMİN ET · **Geri çağırma: dört soru (yanıtlar aşağıda)**
>
> - **S1 (H8).** Kapasiteyi her dolduğunda iki katına çıkaran dinamik diziye (ilk kapasite 1) 10^5 öğe sona eklenince toplam kaç öğe kopyalanır? Öğe başına?
> - **S2 (H9).** `i % k` ile `i & (k - 1)` hangi k değerlerinde her i ≥ 0 için aynıdır? k = 6'da 0 … 999 999 arasındaki indislerin kaçında farklıdır?
> - **S3 (H1).** C'de `-7 % 2` kaçtır? Python'da?
> - **S4 (AP-1 H13).** `strlen("kılıç")` kaç döndürür?

Yanıtlar. S1: 131 071 kopya, öğe başına 1,311 (H8 Örnek 2): toplam 2n'yi aşmaz; yeniden boyutlandırmanın amortize O(1) maliyeti aynı geometrik toplamdan çıkar. S2: yalnızca k 2'nin kuvveti iken; k = 6'da 666 668 indiste farklıdır (H9 Kavram 5). S3: C'de -1, Python'da 1 (H1 Örnek 6): C'de kalan bölünenin, Python'da bölenin işaretini alır; negatif karmadan `%` ile alınan indis C'de negatif olabilir (Hata Avcısı 1). S4: 8 ("kılıç" 5 harf ama 8 bayt: "ı" ve "ç" UTF-8'de ikişer bayttır; Örnek 11): bu haftanın karma fonksiyonları **bayt** üzerinde çalışır.

3. Kavramsal Anlatım: Anahtardan İndise

3.1 Sözlük soyut veri tipi

Sözlük (dictionary, map) anahtar–değer çiftlerini tutan ve üç işlem sunan bir soyut veri tipidir (ADT): **ekle**(anahtar, değer), **ara**(anahtar) ve **sil**(anahtar). Ekle **güncelleme semantiği** taşır: anahtar varsa

değeri güncellenir, böylece bir anahtar en çok bir kez bulunur. Yalnız anahtarları tutan biçim **kümedir** (set): "daha önce gördüm mü?" yapısı. Tablo 10.1 dört gerçekleştirimi karşılaştırır.

Tablo 10.1
Sözlük ADT'sinin dört gerçekleştirimi ve işlem maliyetleri (n: anahtar sayısı)

Gerçekleştirim	ara	ekle	sil	Koşul / not
Sırasız dizi	Θ(n)	Θ(n) (yineleme denetimi)	Θ(n)	her türlü anahtar; yalnızca eşitlik
Sıralı dizi + ikili arama (H4)	Θ(log n)	Θ(n) (kaydırma)	Θ(n)	anahtarlar sıralanabilir; sıralı dolaşma bedava
Doğrudan adresleme	Θ(1)	Θ(1)	Θ(1)	anahtar küçük bir tamsayı evreninden (0 … U − 1)
Karma tablosu	O(1) beklenen	O(1) beklenen (amortize)	O(1) beklenen	eşitlik + karma fonksiyonu; en kötü durum Θ(n)

3.2 Doğrudan adresleme ve sınırları

Anahtar 0 … U − 1 aralığında bir tamsayıysa en basit sözlük, U hücreli bir dizidir: anahtar **doğrudan indistir** (direct addressing). Her işlem tek bir dizi erişimidir. Örnek 1 0–99 numaralı öğrencileri böyle saklar; boş hücre C'de `NULL`, C++'ta boş `std::string`, Python'da `None`:

`c/H10_01_dogrudan_adresleme.c · satır 24–40`

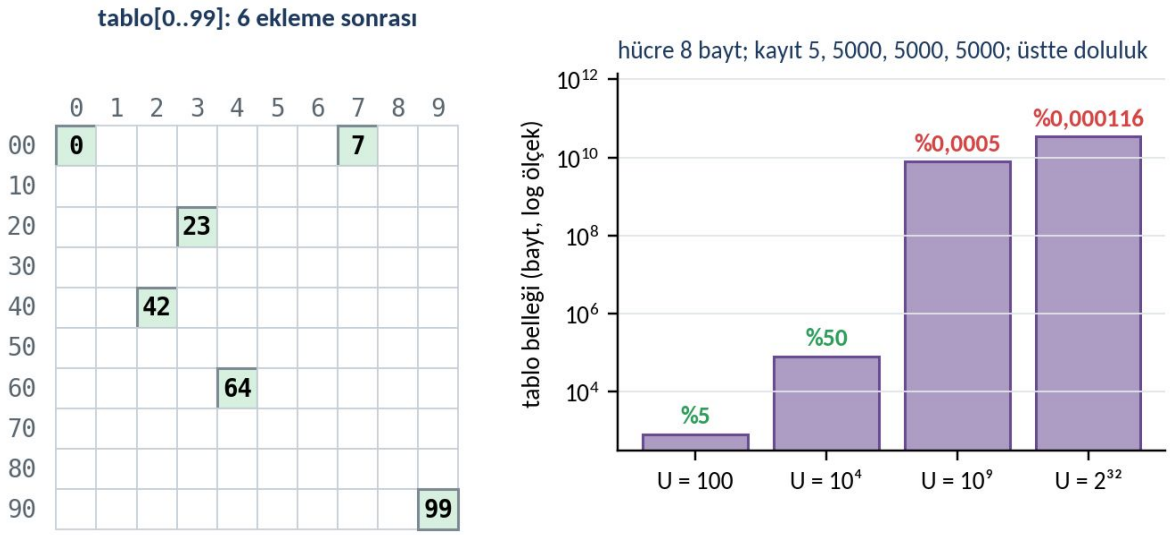
```c
static void ekle(int no, const char *ad)
{
    erisim_sayisi++;
    tablo[no] = ad;              /* anahtar = indis: arama yok */
}

static const char *ara(int no)
{
    erisim_sayisi++;
    return tablo[no];
}

static void sil(int no)
{
    erisim_sayisi++;
    tablo[no] = NULL;
}
```

Çıktı: `ara(42): Zeynep Demir [erişim = 1]`, `toplam erişim: 10 (6 ekleme + 3 arama + 1 silme; ara(150) tabloya erişmez)`. Sınır, **anahtar evreninin** büyüklüğüdür: 9 haneli öğrenci numaraları için tablo 10^9 hücre ister (hücre başına 8 baytlık işaretçiyle 8 GB) ve 5000 kayıtla doluluk %0,0005 olur; 32 bitlik anahtarlarda 34,36 GB (Şekil 10.2). Tablo **kayıt sayısıyla** orantılı olmalıdır. Köprü: numaranın 101'e bölümünden kalanı indis yaparsak tablo 101 hücreye iner, ama `240101007 mod 101 = 70` ve `240101108 mod 101 = 70`: iki farklı numara aynı hücreye düşer.

Şekil 10.2
Doğrudan adresleme (Örnek 1): solda 100 hücreli tablo (anahtar = indis), sağda anahtar evreni büyüdükçe tablo belleği (logaritmik ölçek) ve doluluk



3.3 Karma fonksiyonları ve düzgün dağılım

Karma fonksiyonu h anahtarı 0 … m − 1 aralığında bir kovaya çevirir; α = n/m **yük faktörüdür** (load factor). İyi bir karma fonksiyonu (i) **belirlenimli** (aynı anahtar, aynı kova), (ii) **hızlı** (anahtar uzunluğuyla doğrusal) ve (iii) **düzgün** olmalıdır: her anahtar öbürlerinden bağımsız olarak her kovaya 1/m olasılıkla düşüyormuş gibi davranmalı (**düzgün karma varsayımı**; Cormen vd., 2022). Dört yöntem:

- **Bölme yöntemi:** h(k) = k mod m; m bir **asal** sayı seçilir (Örnek 1'de 101, Örnek 3'te 11, Örnek 2'de 1009 ve 10007). m = 2^p seçilirse yalnız k'nin düşük p biti kullanılır (Bölüm 3.5).
- **Çarpma yöntemi:** h(k) = ⌊(A·k mod 2^{32}) / 2^{32-p}⌋ (çarpımın **üst** p biti), A = 2654435769 ≈ 2^{32}·(√5 − 1)/2 (Knuth, 1998); üst bitler bütün anahtar bitlerinden etkilendiği için m = 2^p olabilir.
- **Polinom karma (Horner):** h = ((b_0·31 + b_1)·31 + b_2)… mod 2^{32}; baytların **sırasını** görür (Rabin–Karp'ın kayan karması bunun mod q biçimidir).
- **FNV-1a:** h_0 = 2166136261, her bayt için h ← (h XOR b)·16777619 (mod 2^{32}); Fowler ve Vo'nun 1991 fikrini Noll geliştirdi (Noll, t.y.).

c/H10_02_karma_fonksiyonlari.c · satır 33–52

```c
/* Polinom (Horner, taban 31): h = 31·h + bayt; uint32_t mod 2^32 sarar */
static uint32_t karma_polinom(const char *s)
{
    uint32_t h = 0;
    for (; *s != '\0'; s++) {
        h = 31u * h + (unsigned char)*s;
    }
    return h;
}

/* FNV-1a 32 bit (Fowler, Noll, Vo): önce XOR, sonra FNV asalıyla çarp */
static uint32_t karma_fnv1a(const char *s)
{
    uint32_t h = 2166136261u;           /* FNV başlangıç değeri */
```

```c
    for (; *s != '\0'; s++) {
        h ^= (unsigned char)*s;
        h *= 16777619u;                 /* FNV asalı (2^24 + 2^8 + 0x93) */
    }
    return h;
}
```

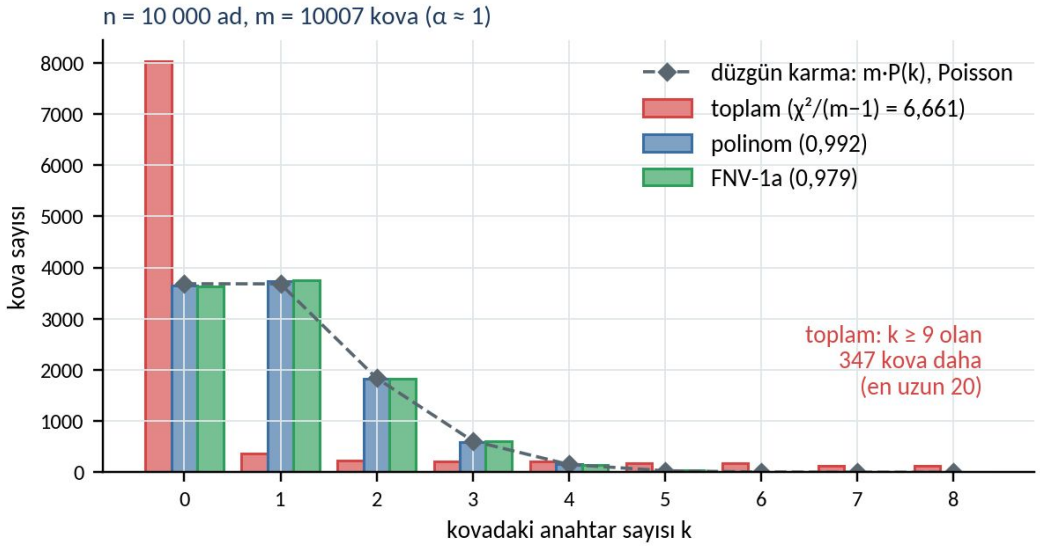
Toplam karma sırayı görmez: “Ali Can”, “Can Ali” ve “Nil Aca” baytlarının toplamı aynıdır (584; polinom ve FNV-1a üçünü ayırır); ayrıca dar bir aralıkta kalır: 10 000 adın toplam karması yalnızca [575, 3531] aralığındadır. Örnek 2 üç fonksiyonun 10 000 ad üzerindeki dağılımını ölçer (Tablo 10.2): boş kova (düzgünde m·e^(−α)), en uzun kova, başarılı aramanın ortalama karşılaştırması Σ c_j(c_j + 1)/2 / n (düzgünde ≈ 1 + α/2) ve **χ^2 ölçütü** Σ (c_j − n/m)²/(n/m): düzgünde $\chi^2/(m-1) \approx 1$, büyük değer kümelenmedir.

Tablo 10.2
Örnek 2: 10 000 adın m = 10 007 kovaya dağılımı (“düzgün” satırı kuramsal beklenti)

m	işlev	boş kova	en uzun	başarılı ort.	χ^2	$\chi^2/(m-1)$
10 007	toplam	8040	20	4,330	66645,6	6,661
10 007	polinom	3655	6	1,496	9923,9	0,992
10 007	FNV-1a	3630	7	1,489	9795,8	0,979
10 007	düzgün	3683,9	-	1,500	10006,0	1,000

m = 10 007'de (α ≈ 1) polinom ve FNV-1a düzgün karmadan ayırt edilemez ($\chi^2/(m-1) \approx 1$, boş kova ≈ m·e^(−α) = 3683,9); toplam karmada 8 040 kova (%80) boş kalır ve bir kovada 20 ad toplanır. Düzgün karmada k anahtarlı kova sayısı Poisson dağılımına uyar (Şekil 10.3). m = 1009'da (α ≈ 9,9) bile toplam karmanın $\chi^2/(m-1)$ değeri 1,672, polinomunki 0,985'tir.

Şekil 10.3
Kova dağılım histogramı (Örnek 2, n = 10 000, m = 10 007); kesikli çizgi Poisson beklentisi



3.4 Çakışmanın kaçınılmazlığı: güvercin yuvası ve doğum günü

m kovaya m + 1 anahtar konursa en az bir kovada iki anahtar vardır (**güvercin yuvası ilkesi**, pigeonhole principle); dizgilerin sayısı sınırsız, 32 bitlik karmaların sayısı 2^{32}'dir. Örnek 3'ün 12 adı 11 kovaya dağılır ve kova 6'da üç ad toplanır (Şekil 10.4). Şaşırtıcı olan, çakışmanın tablo dolmadan **çok önce** başlamasıdır. 23 kişilik bir sınıfta iki kişinin doğum gününün aynı olma olasılığı P = 1 − (365/365)(364/365)…(343/365) = 0,507'dir; 57 kişide 0,9901 olur (**doğum günü paradoksu**). Örnek 10 çarpımı üç dilde aynı işlem sırasıyla hesaplar ve Monte Carlo benzetimiyle karşılaştırır (Bölüm 5.4).

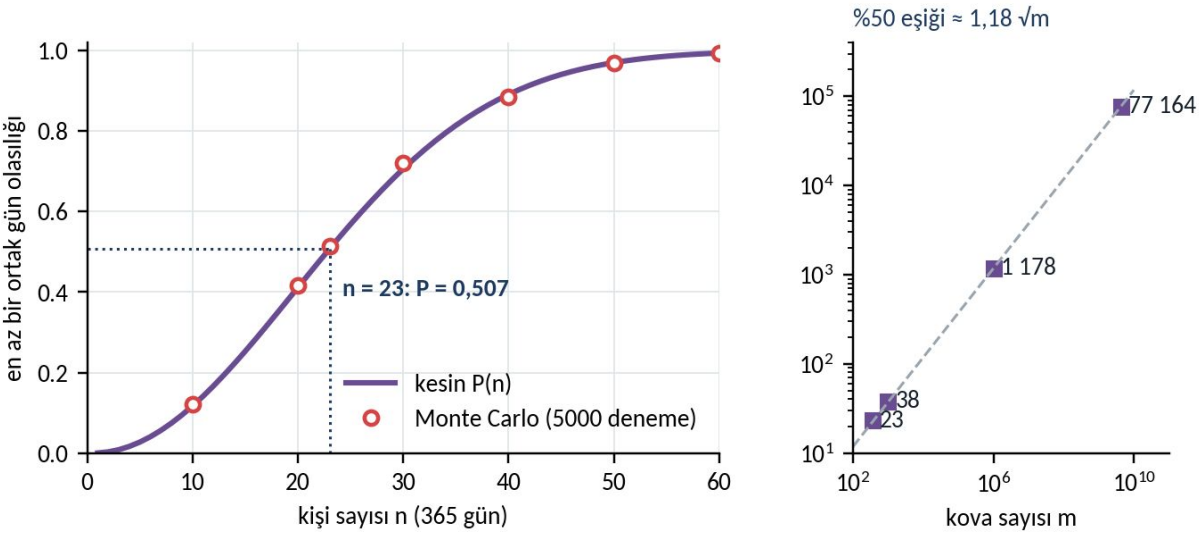
Şekil 10.4
Güvercin yuvası ilkesi (Örnek 3): 12 anahtar, 11 kova; FNV-1a mod 11 ile dört kova boş, kova 6'da üç anahtar



Örnek 3'ün 12 anahtarı (FNV-1a mod 11); boş kova 4, en uzun zincir 3

m kovalı düzgün bir tabloda **ilk çakışma** ortalama ≈ √(πm/2)'nci anahtarda görülür, %50 eşiği ≈ 1,18 √m: m = 365'te 23, m = 10^6'da 1 178, 32 bitlik karmalarda (m = 2^{32}) yalnız 77 164 anahtar (Şekil 10.5). Bu yüzden 32 bitlik karma anahtarın "parmak izi" sayılamaz: tablo anahtarın **kendisini** de saklar ve karma eşitliğinden sonra anahtarları karşılaştırır; Rabin–Karp'ın doğrulama adımı (Bölüm 4.5) aynı kuraldır.

Şekil 10.5
Doğum günü paradoksu (Örnek 10): n kişide en az bir ortak gün olasılığı (n = 23'te 0,507) ve Monte Carlo benzetimi; sağda m kovada %50 çakışma eşiği (log–log, kesikli çizgi √(2 ln 2 · m))



3.5 Kaputun Altında: maske, düşük bitler, önbellek ve işaretsiz taşma

KAPUTUN ALTINDA · Mod yerine maske, ama düşük bitlere dikkat

Tamsayı bölmesi toplama ya da bit işleminden çok daha pahalıdır. $m = 2^p$ iken işaretsiz h için `h % m` ile `h & (m − 1)` aynıdır (Kavram 1 bunu 10^6 değerde m = 16 ve 1024 için denetler); derleyici sabit 2'nin kuvvetine bölmeyi zaten maskeye çevirir, m çalışma anında değişiyorsa bölme kalır. Kavram 1'de $2{\cdot}10^8$ indis hesabı bu çalıştırmada `%` ile 970,5 ms, `&` ile 141,6 ms sürdü.

Maske yalnızca **düşük p biti** görür: h(k) = k ile 1024 düzenli anahtarda m = 16 maske ardışık anahtarlarda 16 kova, çift anahtarlarda 8, 16'nın katlarında yalnızca 1 kova kullanır; asal m = 17 her durumda 17 kovayı, çarpma yönteminin üst 4 biti 16 kovayı doldurur (Şekil 10.6). Düzenli anahtarlar (numaralar, hizalı adresler) yaygın olduğundan 2'nin kuvveti kapasiteli tablolar düşük bitleri iyi karıştıran bir karma (FNV-1a, çarpmanın üst bitleri) ya da asal kapasite kullanır.

KAPUTUN ALTINDA · Bitişik hücreler, dağınık düğümler

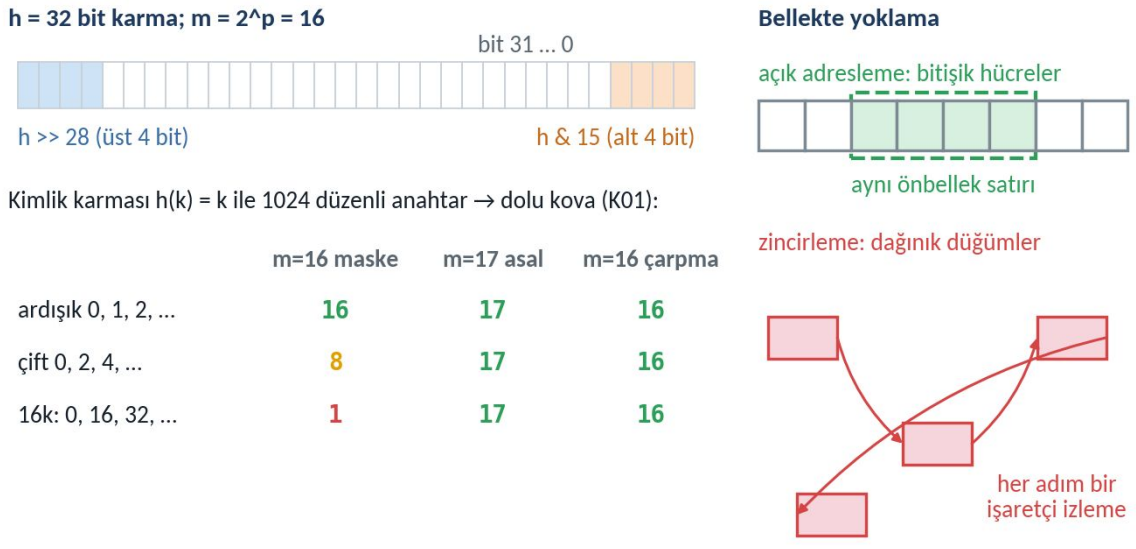
Açık adreslemede yoklama bitişik hücrelerde ilerler: bir önbellek satırı (64 bayt) birkaç hücreyi birden getirir. Zincirlemede her adım ayrı `malloc` edilmiş bir düğüme işaretçi izlemektir; düğüm önbellekte değilse her adım bir bellek erişimi bekler. Fark yoklama **sayısında** görünmez, **sürede** görünür: α ≤ 0,75'te başarılı arama zincirlemede 1,4–2,9 kat yavaştı (Bölüm 6.3; bu makinede, $m = 2^{20}$).

KAPUTUN ALTINDA · İşaretsiz taşma tanımlıdır, işaretli taşma değildir

C ve C++'ta işaretsiz tamsayı aritmetiği mod 2^w yapılır: `uint32_t` ile 31·h + b 2^{32}'yi aşınca taşan kısım tanımlı biçimde atılır (ISO/IEC, 2018). Polinom karma ve FNV-1a bunu bilerek kullanır; Kavram 2 her adımda 64 bitlik "gerçek" değeri ve 32 bitlik sonucu yazar: "karma_tablosu" dizgisinin 6. baytında (116) gerçek değer 97 848 546 405 olur, 32 bitlik sonuç 3 359 265 893'tür: 2^{32}'nin katı atılmıştır. `UINT32_MAX + 1` de tanımlı biçimde 0'dır. **İşaretli** `int` taşması ise tanımsız davranıştır; Hata Avcısı 1'de UBSan bunu yakalar.

Python tamsayısı taşmaz; her adımda `& 0xFFFFFFFF` uygulanınca üç dilin karmaları aynı çıkar (maskesiz değer 67 bite büyür). Yerleşik `hash()` ve `std::hash` çıktıda kullanılmaz (Bölüm 8).

Şekil 10.6
Kaputun Altında: maske düşük, çarpma yöntemi üst bitleri kullanır (Kavram 1); bitişik hücreler ↔ dağınık düğümler



4. Algoritmalar

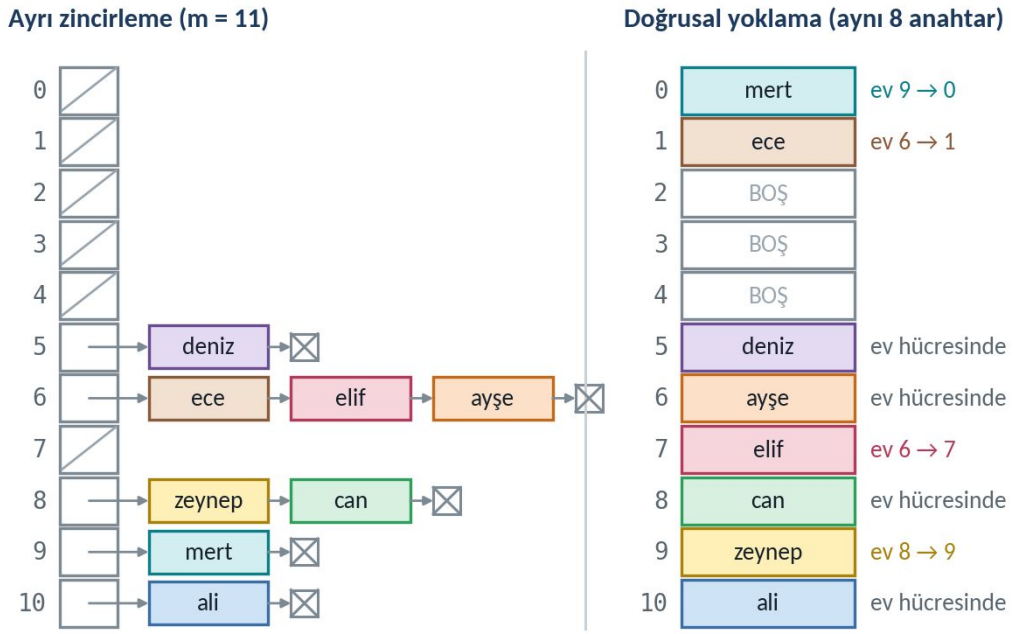
Altı algoritmayı on bir adımlı standart anlatımla (problem, fikir, elle izleme, sözde kod, doğruluk, analiz ve kimlik kartı, C, üç dil, test, deney, varyantlar) işliyoruz: 4.1–4.3 karma tablosunun kendisi, 4.4–4.6 desen eşleme (KMP merak düzeyinde). Karma tablosu örneklerinde aynı adlar ve aynı FNV-1a karması kullanılır; şekillerde her ad aynı renktedir.

4.1 Karma tablosu ve ayrı zincirleme

1 · Problem. n anahtar–değer çiftini (öğrenci adı → not) m kovalı bir tabloda tutmak; ekle, ara ve sil beklenen O(1) sürsün. **2 · Fikir.** Her kova bir **bağlı listedir** (H8): karması j olan anahtarlar j kovasının zincirindedir. Ekleme kovayı bulur, zincirde anahtarı arar, yoksa yeni düğümü zincirin **başına** ekler; arama ve silme yalnızca o zinciri dolaşır (**ayrı zincirleme**, separate chaining). Analoji: arşiv dolapları soyadının baş harfine göredir; bir dosya için tek dolap karıştırılır.

3 · Elle izleme. Örnek 3 12 adı m = 11 kovaya ekler: `ekle elif=90: karma = 0xc0a5c8c3 = 3232090307 -> kova 6`; elif ayşe'nin kovasında zincirin başına geçer, ece gelince zincir ece → elif → ayşe olur (Şekil 10.7, solda). `ara ayşe: 85 (kova 6, karşılaştırma = 3)`; güncelleme: `ekle ayşe=100 -> güncellendi, n = 12`.

Şekil 10.7
Aynı 8 anahtar (FNV-1a mod 11): solda ayrı zincirleme (Örnek 3), sağda doğrusal yoklama (Örnek 4)



SÖZDE KOD · 4 · Sözde kod: ayrı zincirlemede ekle ve sil (ara: EKLE'nin 2–4. satırları)

```
1  algoritma EKLE(T, k, v)                          ▷ T.kova[0..m−1]: zincir başları
2      j ← karma(k) mod m
3      d ← T.kova[j]
4      iken d ≠ BOŞ ve d.anahtar ≠ k:  d ← d.sonraki
5      eğer d ≠ BOŞ ise d.değer ← v;  döndür    ▷ güncelleme
6      yeni düğüm (kopya(k), v, T.kova[j]);  T.kova[j] ← yeni;  n ← n + 1
7  algoritma SİL(T, k)
8      bağ ← &T.kova[karma(k) mod m]            ▷ bağlantının adresi (H8)
9      iken *bağ ≠ BOŞ
10         eğer (*bağ).anahtar = k ise
```

```
11              s ← *bağ;  *bağ ← s.sonraki;  serbest(s);  döndür
12          bağ ← &(*bağ).sonraki
```

5 · Doğruluk. Değişmez (Tablo 10.4): j kovasının listesindeki her anahtar için karma(anahtar) mod m = j; bir anahtar tabloda en çok bir kez bulunur. Ekleme anahtarı yalnız kendi kovasına koyar ve var olanı güncelleyerek "en çok bir kez" koşulunu korur; bu yüzden arama yalnız karma(k) mod m kovasına bakar. Silmede `bag` her zaman "sıradaki düğümü gösteren bağlantının adresidir"; baş düğüm de bir bağlantıyla (`kova[j]`) gösterildiğinden özel durum yoktur (H8 Örnek 5). **6 · Analiz.** Düzgün karma varsayımında zincirin beklenen uzunluğu α = n/m'dir: başarısız arama α, başarılı arama ≈ 1 + α/2 düğüme bakar; beklenen süre **O(1 + α)**, n = O(m) tutulursa (Bölüm 4.3) O(1). En kötü durumda bütün anahtarlar tek kovadadır: Θ(n). Ek bellek O(n + m).

Tablo 10.3
Karma tablosu (genel) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Karma tablosu: ekle / ara / sil** *(hash table: insert / search / delete)*
Strateji	Anahtar → karma fonksiyonu → kova indisi
Girdi önkoşulu	Anahtarlarda eşitlik ve bir karma fonksiyonu
Zaman (en iyi · ortalama · en kötü)	O(1) · O(1) beklenen · Θ(n)
Ek bellek	O(n)
Özellikler	Süre karma kalitesine ve α'ya bağlı; en kötü durumda tek kova (Bölüm 5.6)
Değişmez / doğruluk	Her anahtar, karmasının belirlediği kovada (açık adreslemede: yoklama dizisinde)
Kütüphane karşılığı	C: yok · C++: `std::unordered_map`, `std::unordered_set` · Python: `dict`, `set`
Ne zaman kullanılır	Eşitlik araması: frekans, yinelenen, iki toplam, anagram (Örnek 7–9)
Ne zaman kullanılmaz	Sıralı dolaşma, aralık sorgusu, en küçük/en büyük gerekince

Tablo 10.4
Ayrı zincirleme kimlik kartı

ALGORİTMA KİMLİK KARTI	**Ayrı zincirleme** *(separate chaining)*
Strateji	Her kova bir bağlı liste (H8 düğümü: `Dugum`, `sonraki`)
Girdi önkoşulu	Karma fonksiyonu
Zaman (en iyi · ortalama · en kötü)	O(1) · O(1 + α) · Θ(n)
Ek bellek	O(n + m)
Özellikler	α > 1 olabilir (Örnek 3); silme kolay; her düğüm ayrı ayırma (Bölüm 6.3)
Değişmez / doğruluk	j kovasındaki her anahtar için karma mod m = j; anahtar en çok bir kez
Kütüphane karşılığı	C: yok · C++: `std::unordered_map` · Python: yok — `dict` açık adreslemelidir
Ne zaman kullanılır	α > 1 olabilecekse, sık silmede, düğüm adresi kararlı kalmalıysa
Ne zaman kullanılmaz	Küçük anahtarla çok sık arama (işaretçi izleme, önbellek kaçırma)

7 · C gerçekleştirimi. Örnek 3'te `zincirde_bul` kovanın zincirini dolaşır. Düğüm anahtarın **kendi kopyasını** tutar (`dizgi_kopyala`: `malloc` + `memcpy`; `strdup` C17'de yoktur; tampona bağlanmak Hata Avcısı 2'dir); düğümler ve kopyalar tablonundur, `tablo_yok_et` hepsini bırakır:

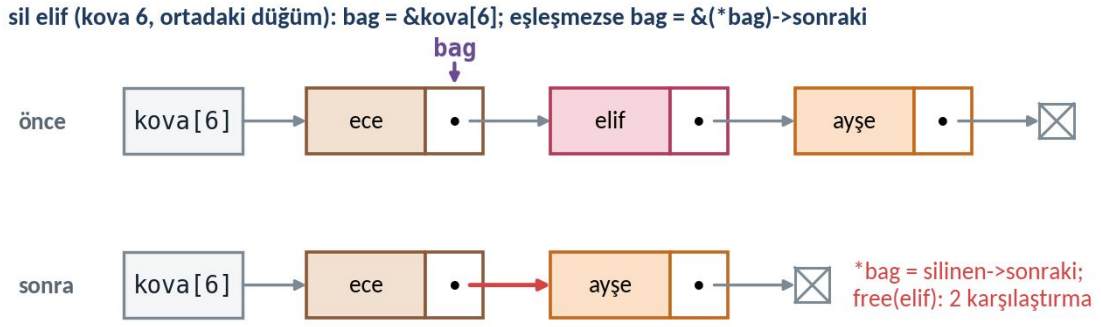
c/H10_03_ayri_zincirleme.c · satır 90–114

```c
/* 1: yeni anahtar, 0: değer güncellendi, -1: bellek yetmedi */
static int tablo_ekle(Tablo *t, const char *anahtar, int deger)
{
    size_t j = kova_no(t, anahtar);
    Dugum *d = zincirde_bul(t->kova[j], anahtar);
    if (d != NULL) {
        d->deger = deger;               /* güncelleme semantiği */
        return 0;
    }
    d = malloc(sizeof *d);
    if (d == NULL) {
        return -1;
    }
    d->anahtar = dizgi_kopyala(anahtar); /* dış tampona bağlanma! */
    if (d->anahtar == NULL) {
        free(d);
        return -1;
    }
    d->deger = deger;
    d->sonraki = t->kova[j];            /* zincirin başına: O(1) */
    t->kova[j] = d;
    t->n++;
    ayrilan_dugum++;
    return 1;
}
```

c/H10_03_ayri_zincirleme.c · satır 127–145

```c
/* Dugum ** ile silme (H8): bağlantıyı tutan işaretçinin adresi gezer */
static int tablo_sil(Tablo *t, const char *anahtar)
{
    Dugum **bag = &t->kova[kova_no(t, anahtar)];
    while (*bag != NULL) {
        karsilastirma_sayisi++;
        if (strcmp((*bag)->anahtar, anahtar) == 0) {
            Dugum *silinen = *bag;
            *bag = silinen->sonraki;
            free(silinen->anahtar);
            free(silinen);
            birakilan_dugum++;
            t->n--;
            return 1;
        }
        bag = &(*bag)->sonraki;
    }
    return 0;
}
```

Şekil 10.8
*Zincirden silme (Örnek 3): bag, silinecek düğümü gösteren bağlantının adresidir; *bag = silinen->sonraki bağı atlatır, sonra free (baştaki düğüm için de aynı kod)*



8 · Üç dil karşılaştırması. C++'ta `std::string` üyesi kopyayı kendisi tutar, düğümler yıkıcıda döngüyle silinir; Python'da `bytes(anahtar)` değişmez bir kopyadır ve silme önceki düğümü izler (Ek A.3):

C++ **H10_03_ayri_zincirleme.cpp · satır 74–80**

```cpp
    Dugum *d = zincirde_bul(t.kova[j], anahtar);
    if (d != nullptr) {
        d->deger = deger;                // güncelleme semantiği
        return 0;
    }
    d = new Dugum{anahtar, deger, t.kova[j]};   // anahtar kopyalanır
    t.kova[j] = d;                       // zincirin başına: O(1)
```

Python **H10_03_ayri_zincirleme.py · satır 63–67**

```python
    d = zincirde_bul(t.kova[j], anahtar)
    if d is not None:
        d.deger = deger                  # güncelleme semantiği
        return 0
    t.kova[j] = Dugum(bytes(anahtar), deger, t.kova[j])  # başa: O(1)
```

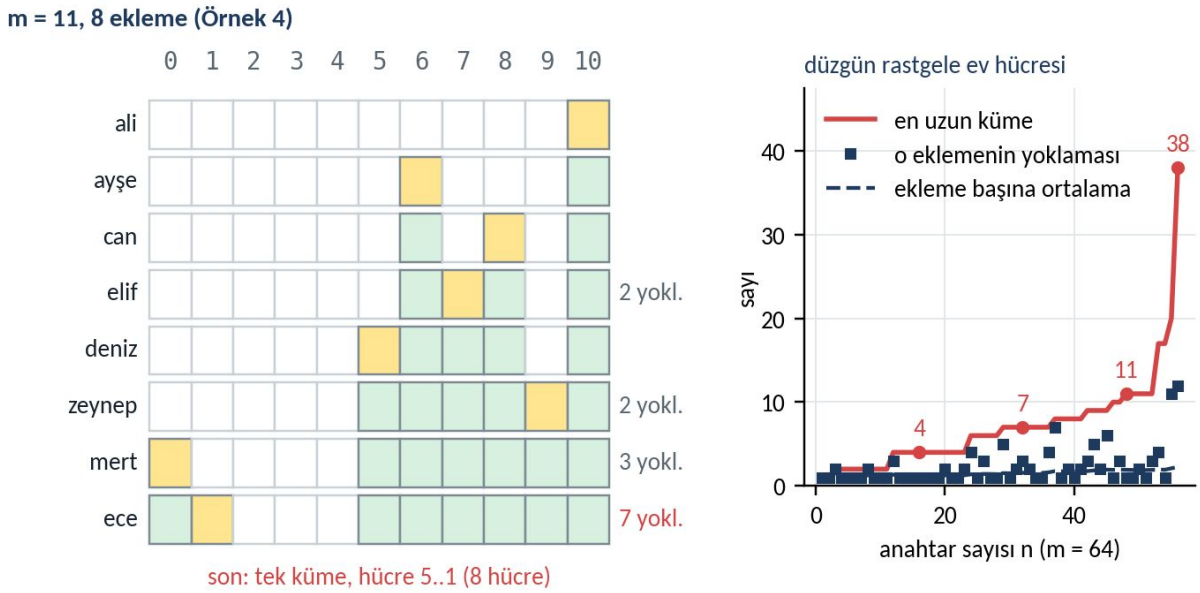
9 · Test. Ortadaki, baştaki, sondaki düğümün ve olmayan anahtarın silinmesi (`sil can: silindi (kova 8, karşılaştırma = 2)`, `sil berk: YOK (kova 4, karşılaştırma = 1)`); sonda `düğüm: ayrılan = 12, serbest bırakılan = 12` ve ASan/Valgrind temiz (kodlar/BELLEK_DENETIMI.txt). **10 · Deney.** m = 2^{16}, α = 0,95: başarılı arama 1,475 (kuram 1,475), başarısız 0,955 düğüm (kuram 0,950; Bölüm 6.2). **11 · Varyantlar.** Sona ekleme, bulunanı başa taşıma (move-to-front), sıralı zincir, uzun zinciri dengeli ağaca çevirme; `std::unordered_map` düğüm tabanlıdır (Bölüm 8).

4.2 Açık adresleme: doğrusal yoklama ve mezar taşı

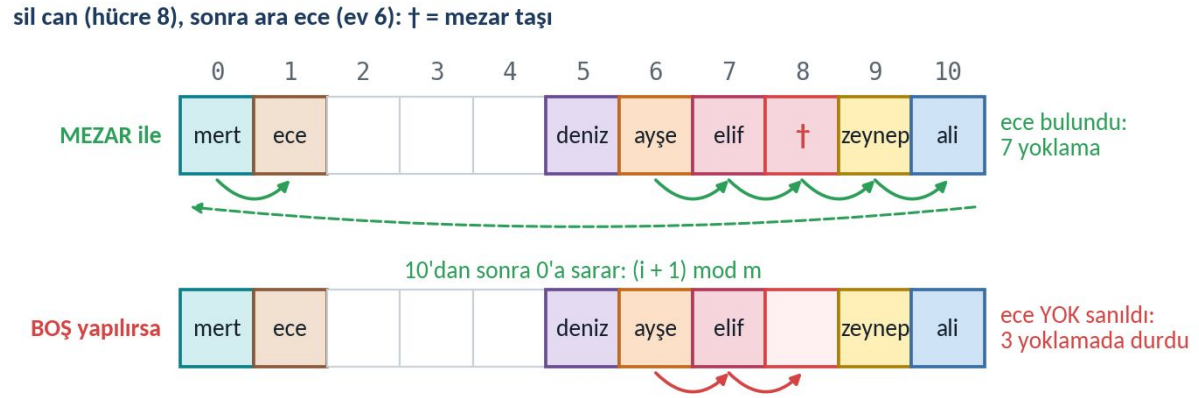
1 · Problem. Aynı sözlüğü düğüm ayırmadan, anahtarları tek bir dizinin hücrelerinde tutarak gerçeklemek (**açık adresleme**). **2 · Fikir.** Anahtarın **ev hücresi** h(k) mod m'dir; doluysa i + 1, i + 2, … (mod m) denenir (**doğrusal yoklama**; bakılan her hücre bir **yoklamadır**). Arama aynı diziyi izler ve **BOŞ** hücrede durur: anahtar tabloda olsaydı o boşluğa ya da daha önceye yerleşirdi. Analoji: numaralı koltuğunuz doluysa sağdaki ilk boş koltuğa oturursunuz; sizi arayan da numaranızdan ilk boş koltuğa kadar bakar.

3 · Elle izleme. Örnek 3'ün ilk sekiz adı m = 11 hücreye (Şekil 10.7, sağda; Şekil 10.9): `ekle elif: ev 6, yoklama 6 7 (2) -> hücre 7`, `ekle mert: ev 9, yoklama 9 10 0 (3) -> hücre 0` (10'dan sonra 0'a sarar), `ekle ece: ev 6, yoklama 6 7 8 9 10 0 1 (7) -> hücre 1`. Son durumda dolu hücreler tek kümedir: `kümeler: [5..1] 8 hücre;`. Bu **birincil kümelenmedir**: kümeye düşen her anahtar kümeyi uzatır ve uzun kümeye düşme olasılığı uzunlukla artar. m = 64'lük benzetimde (düzgün rastgele ev) α = 0,25'te en uzun küme 4, α = 0,75'te 11, n = 56'da 38 hücredir.

Şekil 10.9
Kümelenme: solda Örnek 4'ün sekiz eklemesi (sarı: yerleşilen hücre), sağda m = 64'te en uzun küme ve yoklama



Şekil 10.10
Mezar taşıyla silme (Örnek 4, Hata Avcısı 3): † aramayı durdurmaz (üstte); BOŞ yapılırsa arama 8'de kopar (altta)



Silme ve mezar taşı. Silinen hücreyi BOŞ yapmak aramayı koparır: `can` silinip hücre 8 BOŞ yapılırsa ece (ev 6) aranırken 8'deki boşlukta "yok" çıkar, oysa ece hücre 1'dedir (Hata Avcısı 3). Çözüm hücreyi **MEZAR** (tombstone) yapmaktır: arama mezar taşında durmaz; ekleme ilk mezar taşını yeniden kullanır ama yokluğu BOŞ hücreye kadar doğrular (Şekil 10.10): `ara ece: ev 6, yoklama 6 7 8 9 10 0 1 (7) -> 80`, `ekle kerem: ev 7, yoklama 7 8 9 10 0 1 2 (7) -> hücre 8 (ilk mezar taşı; yinelenen olmasın diye BOŞ hücreye kadar arandı)`.

> **TAHMİN ET · Sizce arama nerede durur?**
>
> Mezar taşlı tabloda (Şekil 10.10, üstte) olmayan umut'u (ev hücresi 5) arıyoruz: arama hangi hücrede durur, kaç yoklama yapar? (Yanıt: mezar taşını geçer, 0'a sarar, ilk BOŞ hücrede durur: `ara umut: ev 5, yoklama 5 6 7 8 9 10 0 1 2 (9) -> YOK (BOŞ hücrede durdu)`.)

SÖZDE KOD · 4 · Sözde kod: doğrusal yoklamada bul ve sil (ekle: BUL; yoksa "yer" hücresine yaz)

```
algoritma BUL(T, k)                          ▷ (k'nin hücresi ya da −1, eklenecek yer)
    i ← h(k) mod m;  ilk_mezar ← −1
    adım ← 0'dan m − 1'e kadar her adım için
        eğer T[i] = BOŞ ise döndür (−1, ilk_mezar ≥ 0 ise ilk_mezar değilse i)
        eğer T[i] = MEZAR ise  ilk_mezar < 0 ise ilk_mezar ← i
        değilse eğer T[i].anahtar = k ise döndür (i, −)
        i ← (i + 1) mod m                    ▷ doğrusal yoklama
    döndür (−1, ilk_mezar)                   ▷ BOŞ hücre yok
algoritma SİL(T, k):  (i, −) ← BUL(T, k);  eğer i ≥ 0 ise T[i] ← MEZAR
```

5 · Doğruluk. Değişmez (Tablo 10.5): **anahtar tablodaysa, ev hücresinden başlayan yoklama dizisinde ilk BOŞ hücreden önce bulunur** (kimlik kartındaki denk ifade: yoklama dizisinde BOŞ hücre yoktur, MEZAR olabilir). Ekleme anahtarı yoklama dizisindeki ilk uygun hücreye koyarak değişmezi kurar; mezar taşı hücreyi "dolu sayılır ama eşleşmez" bıraktığı için silme değişmezi korur, BOŞ yapmak bozar. Döngü en çok m adım sürer (Hata Avcısı 4). **6 · Analiz.** Düzgün karma ve $\alpha < 1$ için Knuth (1998): başarılı arama $\tfrac{1}{2}(1 + 1/(1-\alpha))$, başarısız $\tfrac{1}{2}(1 + 1/(1-\alpha)^2)$ yoklama; $\alpha = 0{,}50$'de 1,5 ve 2,5; 0,75'te 2,5 ve 8,5; 0,95'te 10,5 ve 200,5. Ölçüm ($m = 2^{16}$, 7 tohum): $\alpha = 0{,}75$'te başarısız 8,33, $\alpha = 0{,}95$'te 192,3 yoklama (Bölüm 6.2). Bu yüzden α 1/2–3/4 civarında tutulur. Ek bellek O(m).

Tablo 10.5
Doğrusal yoklama (açık adresleme) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Doğrusal yoklama (açık adresleme)** *(linear probing, open addressing)*
Strateji	Anahtarlar tek dizide
Girdi önkoşulu	$\alpha < 1$ (yük denetimi: tablo dolmadan büyütülür)
Zaman (en iyi · ortalama · en kötü)	O(1) · $\approx \tfrac{1}{2}(1 + 1/(1-\alpha)^2)$ yoklama · Θ(n)
Ek bellek	O(m)
Özellikler	Birincil kümelenme; bitişik bellek önbellek dostudur; silmede mezar taşı
Değişmez / doğruluk	Ev hücresinden anahtarın hücresine dek yoklama dizisinde BOŞ yok
Kütüphane karşılığı	C: yok · C++: yok · Python: `dict`/`set` açık adreslemelidir
Ne zaman kullanılır	Küçük anahtar/değer ve sık arama, $\alpha \le$ ~0,7: bitişik bellek önbellek dostudur
Ne zaman kullanılmaz	$\alpha \to 1$, çok silme (mezar taşı birikimi), kötü karma fonksiyonu

7 · C gerçekleştirimi. Örnek 4'te `bul` iki sonuç döndürür: anahtarın hücresi (yoksa −1) ve çıktı parametresiyle eklenecek yer:

C c/H10_04_dogrusal_yoklama.c · satır 95–121

```
95  static long bul(const Tablo *t, const char *anahtar, long *yer)
```

```
{
    size_t i = ev(t, anahtar);
    long ilk_mezar = -1;
    yol_n = 0;
    for (size_t adim = 0; adim < t->m; adim++) {
        yoklama_sayisi++;
        if (yol_n < 64) {
            yol[yol_n++] = i;
        }
        const Hucre *h = &t->hucre[i];
        if (h->durum == BOS) {
            *yer = ilk_mezar >= 0 ? ilk_mezar : (long)i;
            return -1;
        }
        if (h->durum == MEZAR) {
            if (ilk_mezar < 0) {
                ilk_mezar = (long)i;
            }
        } else if (strcmp(h->anahtar, anahtar) == 0) {
            return (long)i;
        }
        i = (i + 1) % t->m;                  /* doğrusal yoklama */
    }
    *yer = ilk_mezar;                        /* BOŞ hücre hiç yok */
    return -1;
}
```

8 · Üç dil karşılaştırması. `bul`'un ikinci sonucu C'de çıktı parametresi (`long *yer`), C++'ta başvuru (`long& yer`), Python'da demettir. Silme (C'de `tablo_sil`, 162–175. satırlar) üç dilde aynı atamalardır:

C++ **H10_04_dogrusal_yoklama.cpp · satır 128–135**

```
    if (i < 0) {
        return false;
    }
    t.hucre[i].anahtar.clear();
    t.hucre[i].durum = MEZAR;
    t.dolu--;
    t.mezar++;
    return true;
```

Python **H10_04_dogrusal_yoklama.py · satır 100–106**

```
    if i < 0:
        return False
    t.hucre[i].anahtar = None
    t.hucre[i].durum = MEZAR
    t.dolu -= 1
    t.mezar += 1
    return True
```

9 · Test. Program Hata Avcısı 3'ün karşı senaryosunu sınar ve mezar taşlarının **birikimini** ölçer: m = 1024, 512 anahtarlı tabloda her turda 512 sil + 512 ekle; dolu hücre sayısı aynı kalır ama başarısız arama 2,510 yoklamadan 405,959'a çıkar; aynı m ile **yeniden kurma** (yalnız DOLU hücreler) 2,415'e

indirir (son turda mezar taşı 510, (dolu + mezar)/m = 0,998). Yük denetimi bu yüzden (dolu + mezar)/m ile yapılır.

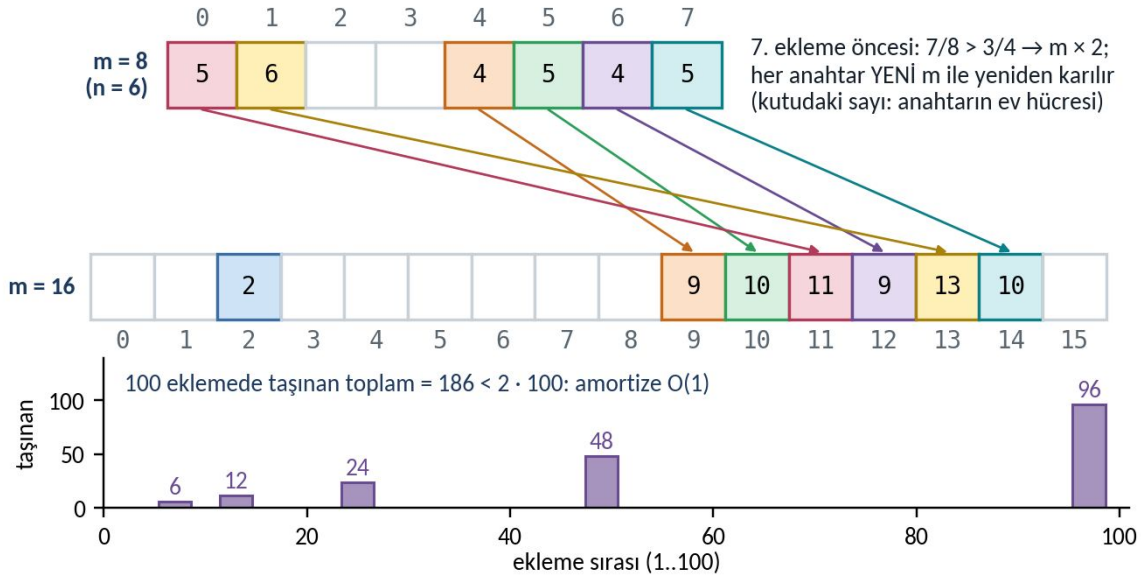
10 · Deney. Örnek 5 ve Haftanın Deneyi (Bölüm 6.2) formülleri ölçümle sınar. **11 · Varyantlar.** Karesel yoklama (i + j²) ve çift karma (adım ikinci bir karmadan) birincil kümelenmeyi kırar; Robin Hood yoklaması ev uzaklığını dengeler; mezarsız **geri kaydırmalı silme** (Alıştırma 10.12; Knuth, 1998, Algoritma 6.4R) birikimi ortadan kaldırır. CPython `dict`'i de açık adreslemeli ve mezar taşlıdır (Bölüm 8).

4.3 Yük faktörü ve yeniden boyutlandırma

1 · Problem. n önceden bilinmiyorken α'yı sınırlı tutmak. **2 · Fikir.** H8'in dinamik dizisindeki gibi kapasiteyi **iki katına** çıkarmak; ama dinamik dizide öğeler aynı indisle kopyalanırken karma tablosunda indis m'ye bağlıdır, bu yüzden her anahtar yeni tabloya **yeniden karılarak** yerleşir (rehash). Örnek 6 (n + 1)/m > 3/4 olacaksa önce büyütür.

3 · Elle izleme. m = 8'den başlayan tabloya ortak LCG'nin anahtarları eklenir: `ekleme #7 öncesi: α = 7/8 > 3/4 olacaktı -> m 8 -> 16, taşınan 6`. Her anahtar yeni m ile yeni ev hücresine gider (Şekil 10.11); 100 eklemenin sonunda: `m = 256, α = 0.391, toplam taşınan = 186`.

Şekil 10.11
Yeniden boyutlandırma (Örnek 6): m = 8 → 16'da yeni ev hücreleri; altta 100 eklemede taşınan anahtarlar



SÖZDE KOD · 4 · Sözde kod: yük denetimli ekleme ve büyütme

```
algoritma EKLE(T, k)
    eğer 4·(T.n + 1) > 3·T.m ise BÜYÜT(T)          ▷ α ≤ 3/4 korunur (tamsayıyla)
    YERLEŞTİR(T, k)
algoritma BÜYÜT(T)
    Y ← boş tablo, kapasite 2·T.m
    i ← 0'dan T.m − 1'e kadar her i için
        eğer T[i] dolu ise YERLEŞTİR(Y, T[i].anahtar)  ▷ YENİ m ile yeniden karılır
    T ← Y                                          ▷ eski dizi serbest bırakılır
```

5 · Doğruluk. Değişmez (Tablo 10.6): Her eklemeden sonra n/m ≤ 3/4; o ana dek taşınanların toplamı < 2N. Büyütme bütün dolu hücreleri taşır, içerik değişmez; yerleşim yeni m'ye göre yapıldığından arama değişmezi yeni tabloda yeniden kurulur. **6 · Analiz. Toplama yöntemi**: m → 2m büyütmesi 3m/4

anahtar taşır; N eklemede taşınanlar 6 + 12 + 24 + … + L geometrik dizisidir, toplamı < 2L ≤ 2N: N eklemenin işi O(N), ekleme başına **amortize O(1)**; büyütmeyi tetikleyen tek ekleme Θ(n). N = 100 000: 15 büyütme, son m = 262 144, taşınan 196 602 (N başına 1,966 < 2), en pahalı tek ekleme 98 305 yazma.

Tablo 10.6
Yeniden boyutlandırma kimlik kartı

ALGORİTMA KİMLİK KARTI	**Yeniden boyutlandırma** *(resizing, rehashing)*
Strateji	Eşik aşılacaksa kapasite × 2
Girdi önkoşulu	Bir yük eşiği (Örnek 6: 3/4)
Zaman (en iyi · ortalama · en kötü)	— · O(1) amortize · Θ(n) tek işlem
Ek bellek	O(n)
Özellikler	Toplam taşınan < 2N; tek büyütme Θ(n) duraklama (Örnek 6)
Değişmez / doğruluk	Her eklemeden sonra n/m ≤ 3/4; o ana dek taşınanların toplamı < 2N
Kütüphane karşılığı	C: yok · C++: rehash, reserve · Python: dict kendiliğinden büyür
Ne zaman kullanılır	n önceden bilinmiyorsa
Ne zaman kullanılmaz	Tek Θ(n) duraklamanın kabul edilemediği gerçek zamanlı sistemler

7 · C gerçekleştirimi. Örnek 6'da (çarpma yöntemi, m = 2ᵖ) `tablo_ekle` önce `if (4 * (t->n + 1) > 3 * m)` ile tamsayı yük denetimi yapar; `buyut` yeni diziyi ayıramazsa −1 döner ve eski tablo değişmez:

c/H10_06_yeniden_boyutlandirma.c · satır 83–103

```c
static int buyut(Tablo *t)
{
    Tablo yeni;
    if (tablo_kur(&yeni, t->p + 1) != 0) {
        return -1;
    }
    size_t m = (size_t)1 << t->p;
    for (size_t i = 0; i < m; i++) {
        if (t->dolu[i]) {
            yerlestir(&yeni, t->anahtar[i]);
            tasinan_sayisi++;
        }
    }
    if (iz_ac) {
        printf("  ekleme #%zu öncesi: α = %zu/%zu > 3/4 olacaktı -> m %zu -> "
               "%zu, taşınan %zu\n", t->n + 1, t->n + 1, m, m, 2 * m, t->n);
    }
    tablo_yok_et(t);
    *t = yeni;
    return 0;
}
```

8 · Üç dil karşılaştırması. C'de eski diziler `free` edilir ve `*t = yeni`; C++'ta döngüden sonra `t = std::move(yeni)` vektörleri taşır, eski bellek kendiliğinden bırakılır; Python'da yeni listeler bağlanır:

C++ **H10_06_yeniden_boyutlandirma.cpp · satır 68–75**

```cpp
    Tablo yeni(t.p + 1);
    std::size_t m = t.anahtar.size();
    for (std::size_t i = 0; i < m; i++) {
        if (t.dolu[i]) {
            yerlestir(yeni, t.anahtar[i]);
            tasinan_sayisi++;
        }
    }
```

Python **H10_06_yeniden_boyutlandirma.py · satır 60–69**

```python
    yeni = Tablo(t.p + 1)
    m = len(t.anahtar)
    for i in range(m):
        if t.dolu[i]:
            yerlestir(yeni, t.anahtar[i])
            tasinan_sayisi += 1
    if iz_ac:
        print(f"  ekleme #{t.n + 1} öncesi: α = {t.n + 1}/{m} > 3/4 "
              f"olacaktı -> m {m} -> {2 * m}, taşınan {t.n}")
    t.anahtar, t.dolu, t.p = yeni.anahtar, yeni.dolu, yeni.p
```

9 · Test. Program her N için taşınanın 2N'den küçük olduğunu denetler. **10 · Deney.** N 100 kat büyürken öğe başına yazma ve yoklama sınırlı kalır (yazma/N < 3): N = 1 000: yazma/N 2,530, yoklama/N 4,709; N = 10 000: yazma/N 2,228, yoklama/N 4,691; N = 100 000: yazma/N 2,966, yoklama/N 6,209. **11 · Varyantlar.** Kapasite biliniyorsa baştan ayırmak (C++ `reserve`); silmelerden sonra **küçültmek** (α < 1/4 → m/2); tek Θ(n) duraklamanın kabul edilmediği sistemlerde **artımlı** yeniden boyutlandırma. C++ `unordered_map` `max_load_factor() = 1.0` ile başlar (ISO/IEC, 2017); CPython `dict` kapasitenin 2/3'ünü kullanır.

4.4 Naif desen eşleme

1 · Problem. n uzunluklu T metninde m uzunluklu P deseninin bütün kaymalarını (shift) bulmak: T[s..s + m − 1] = P olan her s (bayt indisi). **2 · Fikir.** Her s için deseni soldan sağa karşılaştır; ilk uyuşmazlıkta bir sağa kay (**kaba kuvvet**). **3 · Elle izleme.** T = “abracadabra”, P = “abra” (Şekil 10.12): 8 kaymada 16 karşılaştırma, 2 eşleşme (s = 0 ve 7); çoğu kayma ilk karakterde düşer. Sonuç kâhinle aynıdır.

SÖZDE KOD · 4 · Sözde kod: naif desen eşleme

```
algoritma NAİF-EŞLE(T[0..n−1], P[0..m−1])
    s ← 0'dan n − m'ye kadar her s için
        j ← 0
        iken j < m ve T[s + j] = P[j]:  j ← j + 1
        eğer j = m ise s'yi yaz                ▷ eşleşme
```

Şekil 10.12
Naif desen eşleme (Örnek 11): her kaymada soldan sağa karşılaştırma, ilk uyuşmazlıkta bir sağa kayma; yeşil eşleşen, kırmızı uyuşmayan karakter



5 · Doğruluk. Değişmez: s'den küçük her kayma denenmiş, eşleşenler yazılmıştır. İç döngü j = m ile biterse eşleşmedir, erken biterse değildir. **6 · Analiz.** En kötü durumda (n − m + 1)·m = O(n·m): a^{10000} metninde $a^{99}b$ deseni 990 100 karşılaştırma ister (kayma başına 100). σ harfli düzgün rastgele metinde kayma başına beklenen karşılaştırma ≤ 1/(1 − 1/σ) (σ = 4: 4/3): n = 10^5, m = 8'de 133 295 karşılaştırma, kayma başına 1,333. Ek bellek O(1).

Tablo 10.7
Naif desen eşleme kimlik kartı

ALGORİTMA KİMLİK KARTI **Naif desen eşleme** *(naive string matching)*	
Strateji	Kaba kuvvet: her kayma s için P, T[s..s+m−1] ile soldan sağa karşılaştırılır
Girdi önkoşulu	Yok
Zaman (en iyi · ortalama · en kötü)	O(n) · O(n) beklenen (rastgele metin) · O(n·m)
Ek bellek	O(1)
Özellikler	Rastgele metinde kayma başına ≈ 1–1,33 karşılaştırma (Örnek 11)
Değişmez / doğruluk	s'den küçük her kayma denenmiş, eşleşenler yazılmıştır
Kütüphane karşılığı	C: strstr · C++: std::string::find · Python: bytes.find, in
Ne zaman kullanılır	Kısa desen, tek seferlik arama, rastgele benzeri metin
Ne zaman kullanılmaz	Tekrarlı desen ve metin (aa…ab), en kötü durum güvencesi gerekince

7 · C gerçekleştirimi. Örnek 11'de sayılan işlem karakter karşılaştırmasıdır:

C `c/H10_11_naif_eslesme.c · satır 31–46`

```c
    for (size_t s = 0; s + m <= n; s++) {    /* s: kayma (shift) */
        size_t j = 0;
        while (j < m) {
```

```
            karsilastirma_sayisi++;
            if (t[s + j] != p[j]) {
                break;                      /* ilk uyuşmazlıkta bırak */
            }
            j++;
        }
        if (j == m) {
            if (say < EN_COK) {
                konum[say] = s;
            }
            say++;
        }
    }
```

8 · Üç dil karşılaştırması. Döngü üç dilde aynıdır (Ek A.11); Python'da metin `bytes`'tır ki indis bayt olsun. UTF-8'de bayt ve karakter konumu farklıdır: `bayt 27 = karakter 23`. **9 · Test.** Kâhin: `strstr`, `std::string::find`, `bytes.find` döngüsü (çakışan eşleşmeler dahil). **10 · Deney.** m = 3'te 1 577 eşleşme, 131 188 karşılaştırma (kayma başına 1,312). **11 · Varyantlar.** Rabin–Karp, KMP, Boyer–Moore (sağdan karşılaştırır, uzun desende kayma başına çok karakter atlar).

4.5 Rabin–Karp: kayan karma

1 · Problem. Naif yöntemin pencere başına m karşılaştırmasını **bir sayı karşılaştırmasına** indirmek. **2 · Fikir.** Pencereyi d tabanında bir sayı gibi karıştır: t_s = (T[s]·d^(m−1) + … + T[s+m−1]) mod q; desenin karması p. Pencere kayınca baştaki rakam çıkar, sondaki girer ve yeni karma O(1)'de bulunur (**kayan karma**): t_(s+1) = (d·(t_s − T[s]·h) + T[s+m]) mod q, h = d^(m−1) mod q (Karp & Rabin, 1987; Cormen vd., 2022). Karmalar eşitse pencere **karakter karakter doğrulanır**: farklı pencereler aynı karmayı verebilir (**sahte eşleşme**, spurious hit).

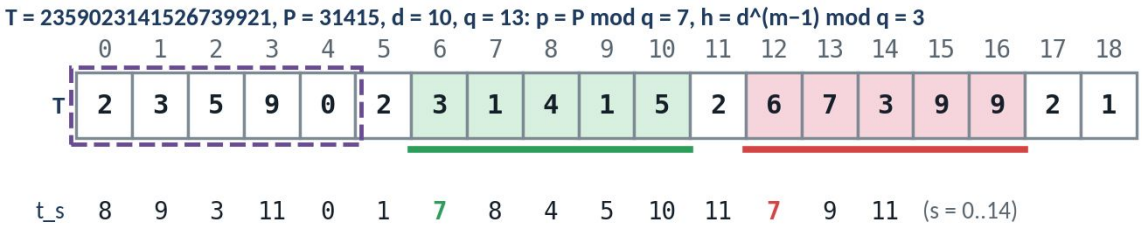
3 · Elle izleme. Cormen vd.'nin (2022) örneği: T = 2359023141526739921, P = 31415, d = 10, q = 13; h = 3, p = 7 (Şekil 10.13). İlk güncelleme: t_1 = (10·(8 + 13 − 6) + 2) mod 13 = 9. Karma eşitliği iki kaymada görülür: s = 6'da pencere 31415 = P (doğrulandı: eşleşme), s = 12'de 67399 ≠ P (sahte eşleşme).

> TAHMİN ET · **Sizce sahte eşleşme var mı?**
>
> q = 13 yerine q = 11 seçilseydi p = 31415 mod 11 = 10 olurdu. T'nin 15 penceresinden kaçının karması 10 çıkar; aralarında sahte eşleşme var mı? (Yanıt: 2 pencere: s = 6 (31415, gerçek eşleşme) ve s = 9 (15267 = 11 · 1387 + 10, sahte). q = 13'te sahte eşleşme s = 12'deydi: sahte eşleşmenin yeri q'ya bağlıdır, bildirilen konumlar değildir, çünkü her aday doğrulanır.)

SÖZDE KOD · 4 · Sözde kod: Rabin–Karp (+ q: ara değer negatif olmasın)

```
algoritma RABİN-KARP(T[0..n−1], P[0..m−1], d, q)
    h ← d^(m−1) mod q;  p ← 0;  t ← 0
    i ← 0'dan m − 1'e kadar her i için
        p ← (d·p + P[i]) mod q;  t ← (d·t + T[i]) mod q        ▷ Horner
    s ← 0'dan n − m'ye kadar her s için
        eğer p = t ise
            eğer T[s..s+m−1] = P ise s'yi yaz  değilse sahte ← sahte + 1
        eğer s < n − m ise t ← (d·(t + q − T[s]·h mod q) + T[s+m]) mod q
```

Şekil 10.13
Rabin–Karp (Örnek 12, d = 10, q = 13): t_s değerleri; s = 6 gerçek (yeşil), s = 12 sahte eşleşme (kırmızı)



Kayan güncelleme (kesikli pencere s = 0 → 1):

```
t_(s+1) = (d · (t_s + q − T[s]·h mod q) + T[s+m]) mod q
t_1 = (10·(8 + 13 − 6) + 2) mod 13 = 9   (çıkan 2, giren 2; O(1) iş)
```

yeşil: s = 6 gerçek eşleşme · kırmızı: s = 12 sahte eşleşme (67399 mod 13 = 7)

5 · Doğruluk. Değişmez (Tablo 10.8): Her kaymada t_s = (Σ T[s+i]·d^(m−1−i)) mod q. Başlatma Horner ile kurar; güncellemede baştaki terim düşer, üsler birer artar, yeni son terim girer: değişmez s + 1 için kurulur. Doğrulama sayesinde bildirilen her konum gerçektir; karma eşitliği yalnız **adaydır** (Hata Avcısı 5). **6 · Analiz.** Ön işlem Θ(m), kayma başına O(1); doğrulama yalnız karma eşitliğinde. Rastgele metinde sahte eşleşme ≈ (n − m + 1)/q olduğundan beklenen süre O(n + m) (q ≥ m); en kötü durum O(n·m) (her pencere eşit karmalı).

Tablo 10.8
Rabin–Karp kimlik kartı

ALGORİTMA KİMLİK KARTI **Rabin–Karp** *(rolling hash)*	
Strateji	Kayan karma: pencerenin karması O(1)'de güncellenir
Girdi önkoşulu	Taban d ve modül q seçimi
Zaman (en iyi · ortalama · en kötü)	O(n + m) · O(n + m) beklenen · O(n·m)
Ek bellek	O(1)
Özellikler	Sahte eşleşme ≈ (n − m + 1)/q, doğrulamayla elenir; q seçimi kritik (Örnek 12)
Değişmez / doğruluk	Her kaymada t_s = (Σ T[s+i]·d^(m−1−i)) mod q
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Aynı uzunlukta çok desen; yinelenen blok ya da intihal tespiti
Ne zaman kullanılmaz	Modül küçük ya da sabitken saldırgan girdi seçebiliyorsa

7 · C gerçekleştirimi. Örnek 12'de q < 2^{31}'dir ve ara değerler `long long`'a sığar. `T[s]·h % q` çıkarılmadan önce **+ q** eklenir: C'de `%` bölünenin işaretini alır, negatif ara değerin kalanı negatif olurdu (H1):

`c/H10_12_rabin_karp.c · satır 60–79`

```c
    long long h = 1;                        /* d^(m-1) mod q */
    for (size_t i = 1; i < m; i++) {
        h = h * d % q;
```

```
    }
    long long hp = 0;
    long long ht = 0;
    for (size_t i = 0; i < m; i++) {          /* Horner */
        hp = (d * hp + p[i]) % q;
        ht = (d * ht + t[i]) % q;
    }
    for (size_t s = 0;; s++) {
        if (hp == ht) {
            dogrula(t, s, p, m, r);
        }
        if (s + m >= n) {
            break;
        }
        /* + q: C'de % bölünenin işaretini alır; ara değer negatif olmasın */
        ht = (d * (ht + q - t[s] * h % q) + t[s + m]) % q;
    }
```

Örnek 12'nin 2. bölümü bunu sayıyla gösterir: s = 2'de x = t_s − T[s]·h = −12; C/C++'ta x % q = −12, Python'da 1; + q ile üç dilde 1.

8 · Üç dil karşılaştırması. C++'ta baytlar `bayt(t, s)` ile `unsigned char`'a çevrilir; Python'da `bytes` öğeleri 0–255'tir ve `%` + q olmadan da doğru sonuç verirdi, ama ifade üç dilde aynı tutulur (Alıştırma 10.13):

C++ **H10_12_rabin_karp.cpp · satır 75–86**

```
    for (std::size_t s = 0;; s++) {
        if (hp == ht) {
            dogrula(t, s, p, r);
        }
        if (s + m >= n) {
            break;
        }
        // + q: C++'ta % bölünenin işaretini alır; ara değer negatif olmasın
        ht = (d * (ht + q - bayt(t, s) * h % q) + bayt(t, s + m)) % q;
    }
    return r;
}
```

Python **H10_12_rabin_karp.py · satır 59–69**

```
    s = 0
    while True:
        if hp == ht:
            dogrula(t, s, p, r)
        if s + m >= n:
            break
        # + q gerekmez (Python'da % bölenin işaretini alır) ama C ile aynı
        # adımlar için korunur
        ht = (d * (ht + q - t[s] * h % q) + t[s + m]) % q
        s += 1
    return r
```

9 · Test. Kâhin naif yöntemdir. **10 · Deney.** n = 10^5 rastgele metin ({a, b, c, d}), m = 8; karma eşitliği / sahte eşleşme: d = 256, q = 13: 7 578 / 7 577; d = 256, q = 1009: 107 / 106; d = 256, q = 1000003: 1 / 0; d = 256, q = $2^{31} - 1$: 27 / 26; d = 31, mod 2^{32}: 1 / 0; d = 256, mod 2^{32}: 406 / 405. q = 13 ile 7 578 karma eşitliğinin 7 577'si sahtedir (≈ kayma/13); q = 1000003 ile sahte kalmaz. İki tuzak: mod 2^{32}'de $256^4 \equiv 0$ olduğundan karmayı yalnız son 4 bayt belirler; q = $2^{31} - 1$'de $2^{31} \equiv 1$ olduğu için baytlar bit konumlarında üst üste biner (`d = 256 = 2^8, q = 2^31 − 1: h(bbdcbbbb) = 656878378, h(aacbdddd) = 656878378`). Karp ve Rabin (1987) q'yu her aramada **rastgele** bir asal seçmeyi önerir: hiçbir girdi önceden kötü bir q'ya göre hazırlanamaz.

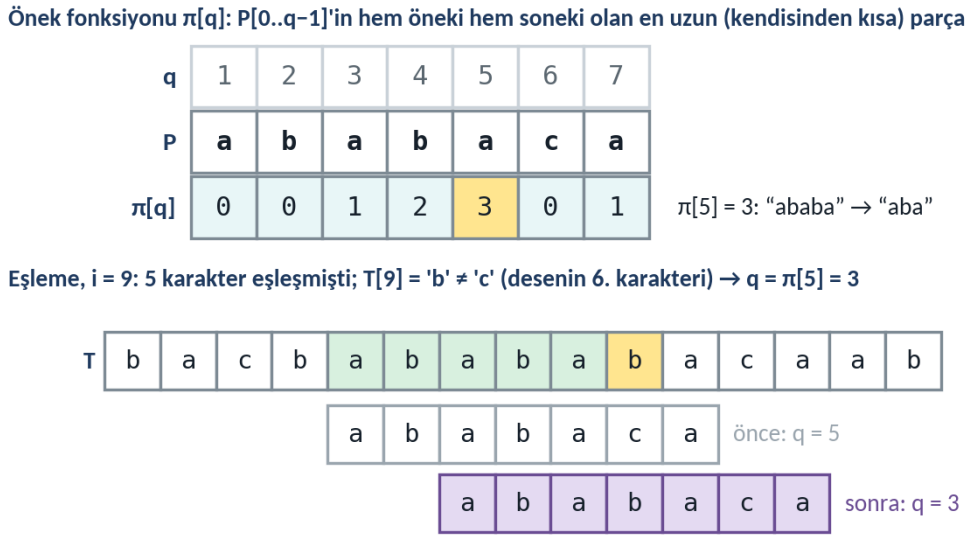
11 · Varyantlar. Aynı uzunlukta k desen (desen karmaları bir karma kümesinde, tek geçiş); iki boyutlu eşleme; intihal tespiti. Mini-OBS 2 v2.9 Rabin–Karp'ı ad–soyad metninde kullanır ve doğal metinde naiften yavaş bulur (v2.9 belgesi § 8.6).

4.6 KMP: önek fonksiyonu (merak)

1 · Problem. En kötü durumda da Θ(n + m) desen eşleme. **2 · Fikir.** Naif yöntem uyuşmazlıkta metinde geri gider; oysa q karakter eşleşmişse T'nin son q karakteri P'nin ilk q karakteridir ve sonraki olası hizalanma yalnız desenden bilinir. **Önek fonksiyonu** π[q], P'nin ilk q karakterinin hem öneki hem soneki olan en uzun özgün parçanın uzunluğudur (Knuth vd., 1977). Uyuşmazlıkta i ilerlemeye devam eder, desende q ← π[q].

3 · Elle izleme. P = ababaca: π = [0, 0, 1, 2, 3, 0, 1]; π[5] = 3, çünkü “ababa”nın en uzun sınırı “aba”dır. T = bacbababababacaab içinde i = 9'daki uyuşmazlıkta q = π[5] = 3'e düşülür, i yerinde kalır (Şekil 10.14); i = 12'de q = 7 = m olur: eşleşme, kayma s = 6.

Şekil 10.14
KMP (Örnek 14): ababaca'nın önek fonksiyonu π ve i = 9'daki uyuşmazlıkta kayma



SÖZDE KOD · 4 · Sözde kod: önek fonksiyonu ve KMP eşlemesi

```
1  algoritma ÖNEK(P[0..m−1])              ▷ pi[q−1] = π[q]
2      pi[0] ← 0;  k ← 0
3      q ← 1'den m − 1'e kadar her q için
4          iken k > 0 ve P[k] ≠ P[q]:  k ← pi[k − 1]
5          eğer P[k] = P[q] ise k ← k + 1
6          pi[q] ← k
7  algoritma KMP(T[0..n−1], P):  q ← 0
```

```
    i ← 0'dan n − 1'e kadar her i için
        iken q > 0 ve P[q] ≠ T[i]:  q ← pi[q − 1]
        eğer P[q] = T[i] ise q ← q + 1
        eğer q = m ise  i − m + 1'i yaz;  q ← pi[q − 1]
```

5 · Doğruluk. Değişmez: q, T[0..i]'nin soneki olan P öneklerinin en uzununun uzunluğudur. T'nin sonekiyle P'nin öneki arasındaki bütün eşleşme uzunlukları π zincirindedir; hiçbir eşleşme atlanmaz (Cormen vd., 2022, Bölüm 32.4). **6 · Analiz.** Her karşılaştırmadan sonra ya i ilerler ya q azalır; toplam ≤ 2n, önek fonksiyonu ≤ 2m: **Θ(n + m)**, ek bellek O(m). Naifin en kötü girdisi: `karşılaştırma: π = 197 (≤ 2m = 200), KMP = 19901 (≤ 2n = 20000), naif = 990100`.

Tablo 10.9
KMP kimlik kartı (merak)

ALGORİTMA KİMLİK KARTI	**KMP (merak)** *(Knuth–Morris–Pratt)*
Strateji	Önek fonksiyonu π: uyuşmazlıkta metinde geri dönülmez, desende q = π[q] yapılır
Girdi önkoşulu	Önek fonksiyonu önceden Θ(m)'de hesaplanır
Zaman (en iyi · ortalama · en kötü)	Θ(n + m) · Θ(n + m) · Θ(n + m)
Ek bellek	O(m)
Özellikler	Metinde geri dönmez; toplam karşılaştırma ≤ 2n + 2m (Örnek 14)
Değişmez / doğruluk	q, T[0..i]'nin soneki olan P öneklerinin en uzununun uzunluğudur
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Akan veri (metinde geri dönülemez), en kötü durum güvencesi gereken aramalar
Ne zaman kullanılmaz	Kısa desen ve rastgele metin: naif yöntem pratikte yeterli

7 · C gerçekleştirimi. Örnek 14'te `onek_fonksiyonu` (34–51. satırlar) sözde kodun ilk yarısıdır; `kmp_ara` (54–95. satırlar) her karşılaştırmayı `esit_mi` ile sayar (C++ ve Python: Ek A.14).

8 · Üç dil. Konum listeleri C'de dizi + sayaç, C++'ta `std::vector`, Python'da liste (Ek A.14). **9 · Test.** KMP, naif, Rabin–Karp ve kâhin dört metinde aynı konumları verir. **10 · Deney.** Rastgele metinde KMP ile naif yakındır (`karşılaştırma: π = 10 (≤ 2m = 16), KMP = 123281 (≤ 2n = 200000), naif = 133295`). **11 · Varyantlar.** Aho–Corasick, sonek dizileri (Bölüm 13).

5. Çözümlü Örnekler ve Uygulamalar

Bu bölümde aynı dört kalıp tekrar eder: **sayaç** (anahtar → kaç kez), **küme** (var/yok), **harita** (anahtar → ilk konum ya da grup) ve **anahtar tasarımı** (eşit sayılacak nesnelere aynı anahtar). Her örnek sonucu bir **kâhinle** (kaba kuvvet ya da kütüphane) karşılaştırır; $O(n^2) \to O(n)$ iddiası sayaçlarla ölçülür.

5.1 Kelime frekansı (Örnek 7)

Problem. Bir metindeki her sözcüğün kaç kez geçtiğini bulmak ve en sık 20'sini yazmak. Girdi Ömer Seyfettin'in üç öyküsüdür (“Diyet”, “Ferman” ve “Forsa”; Seyfettin, 1918/2022, 1917/2023, 1919/2024; toplam 52 424 bayt). Yazar 6 Mart 1920'de ölmüştür; 5846 sayılı Fikir ve Sanat Eserleri Kanunu'nun 27. maddesine göre koruma süresi ölümden itibaren 70 yıl olduğundan metinler kamu malıdır (Polat, 2007; ayrıntı ve Vikikaynak sürümleri `kodlar/girdiler/KAYNAK_NOTU.txt`).

Çözüm. Sözcük, "harf baytlarının en uzun dizisi"dir: ASCII harfleri ve UTF-8 çok baytlı karakterlerin baytları (≥ 0x80). Büyük harf **elle** küçültülür (A–Z → +32); Türkçe büyük harfler olduğu gibi kalır (Bölüm 8). Sayaç (`say`), Örnek 3'teki gibi ayrı zincirlemeli bir karma tablosudur: sözcük varsa değeri artar, yoksa 1 ile eklenir; α > 1 olunca tablo iki katına çıkar. Sözcük ayırma döngüsü:

`c/H10_07_kelime_frekansi.c · satır 196–211`

```c
    for (size_t i = 0; i <= boyut; i++) {
        unsigned char b = i < boyut ? metin[i] : ' ';
        if (harf_mi(b)) {
            if (uzunluk < sizeof sozcuk - 1) {
                sozcuk[uzunluk++] =
                    (char)(b >= 'A' && b <= 'Z' ? b + 32 : b);
            }
        } else if (uzunluk > 0) {
            sozcuk[uzunluk] = '\0';
            if (say(t, sozcuk) != 0) {
                return 1;
            }
            toplam++;
            uzunluk = 0;
        }
    }
```

Program 6 248 sözcükte 3 277 farklı sözcük bulur; son tablo m = 4 096 kova, α = 0,800, en uzun zincir 5 (büyütmelerde toplam 4 088 düğüm taşındı):

ÇIKTI

```
$ ./H10_07_kelime_frekansi
En sık 20 sözcük (sıklık azalan; eşitlikte bayt sırası artan)
   1.   180  bir
   2.    67  bu
   3.    50  gibi
   …
Türkçe büyük harfle (Ç Ğ İ Ö Ş Ü) başlayan farklı sözcük: 56 (ASCII dışı harfler
küçültülmedi)
```

En sık 20'yi yazmak için kütüphane sıralaması kullanılır (`qsort`, `std::sort`, `list.sort`; çıktı biçimidir, algoritma değil). Kütüphane karşılığı (Kavram 7: `std::unordered_map`, `dict`) aynı 20 satırı verir; süreler Bölüm 6.5'te.

5.2 Yinelenen tespiti ve iki toplam (Örnek 8)

Yinelenen tespiti. H2'de (Mini-OBS 2 v2.1) bütün çiftleri, n(n − 1)/2 işlemle karşılaştırmıştık. Karma kümesinde her numara bir kez yoklanır (görüldüyse sayaç artar); eşit çift sayısı Σ c(c − 1)/2'dir. n = 1000'de kaba kuvvet 499 500 karşılaştırma, karma 1 000 yoklama yapar ve ikisi aynı sonucu verir; n = 10^5'te karma 107 133 yoklama yapar (numara başına 1,071; Şekil 10.15).

Şekil 10.15
Yinelenen tespiti ve iki toplam: kaba kuvvet bütün çiftleri, karma kümesi tek geçişte her öğeyi bir kez yoklar (Örnek 8)



İki toplam (two-sum): a[i] + a[j] = T olan bir çift var mı? Kaba kuvvet Θ(n²)'dir. Tek geçişte j ilerlerken **önce** T − a[j] haritada aranır, **sonra** a[j] eklenir; önce eklemek a[j] + a[j] = T'de öğeyi kendisiyle eşler (Alıştırma 10.8). Harita yalnız **ilk** konumu tuttuğundan bulunan çift kaba kuvvetinkiyle aynıdır:

C `c/H10_08_yinelenen_iki_toplam.c · satır 194–212`

```c
    int bulundu = 0;
    for (size_t j = 0; j < n; j++) {
        int var;
        if (a[j] <= T) {
            size_t yer = harita_yer(&h, T - a[j], &var);
            if (var) {
                *bi = (size_t)h.deger[yer];
                *bj = j;
                bulundu = 1;
                break;
            }
        }
        size_t yer = harita_yer(&h, a[j], &var);
        if (!var) {                          /* yalnız ilk konum tutulur */
            h.dolu[yer] = 1;
            h.anahtar[yer] = a[j];
            h.deger[yer] = (int)j;
        }
    }
```

ÇIKTI

```
$ ./H10_08_yinelenen_iki_toplam
2) İki toplam: a[i] + a[j] = T olan ilk çift (en küçük j, sonra en küçük i)
  küçük, n = 7, T = 9: (3, 4) 7 + 2 | karma yokl=8 | kaba kar=10, aynı mı? evet
  …
  tek T, n = 100000, T = 999999: çift yok | karma yokl=239667
  son ikili, n = 100000, T = 1105010: (665, 953) 813292 + 291718 | karma yokl=1910
```

Çözüm yoksa kaba kuvvet n = 10^5'te 4 999 950 000 karşılaştırma yapardı; karma 239 667 yoklama yapar. C/C++'ta 2 400 000 000 + i numaraları `uint32_t`'dir ve toplam 64 bitte hesaplanır; Python tamsayısı taşmaz.

5.3 Anagram gruplama: anahtar tasarımı (Örnek 9)

Anagramlar aynı harflerin farklı dizilişleridir (kitap – takip). Yüzlerce sözcüğü gruplamanın anahtarı, sözcüğün **sıralanmış baytlarıdır** (H7'nin sayma sıralaması, Θ(L + 256)); karma tablosu anahtar → üye listesi haritasıdır:

c/H10_09_anagram_gruplama.c · satır 56–69

```c
static void anahtar_yap(const char *sozcuk, char *anahtar)
{
    int sayac[256] = {0};
    for (const char *s = sozcuk; *s != '\0'; s++) {
        sayac[(unsigned char)*s]++;
    }
    size_t k = 0;
    for (int b = 1; b < 256; b++) {
        for (int c = 0; c < sayac[b]; c++) {
            anahtar[k++] = (char)b;
        }
    }
    anahtar[k] = '\0';
}
```

ÇIKTI

```
$ ./H10_09_anagram_gruplama
  kitap: bayt 6b 69 74 61 70 -> anahtar 61 69 6b 70 74
  takip: bayt 74 61 6b 69 70 -> anahtar 61 69 6b 70 74
   1. (3) akar arka kara
  …
  tek üyeli: 4 sözcük: defter elma okul silgi
  anahtar karşılaştırması = 21 (m = 31 kova)
  (bayt sırası: "maç" < "çam", çünkü 'm' = 0x6d < 0xc3 = 'ç'nin ilk baytı)
```

16 grup ve 4 tek üyeli sözcük çıkar. Anahtar yalnız eşitlik içindir (“kış”ın anahtarı okunamaz baytlardır). Python'da anahtar `bytes`'tır; `sorted(s)` **liste** döndürür ve liste anahtar olamaz (Hata Avcısı 6).

5.4 Doğum günü simülasyonu (Örnek 10)

Örnek 10, Bölüm 3.4'teki kesin P(n)'yi **Monte Carlo** ile de tahmin eder: “bu gün görüldü mü?” 365 hücrelik bir **doğrudan adresleme** tablosudur; her denemede sıfırlamak yerine hücreye deneme numarası (damga) yazılır:

c/H10_10_dogum_gunu.c · satır 33–43

```c
static int deneme_yap(int n, int *gorulen, int damga)
{
    for (int k = 0; k < n; k++) {
        int g = rastgele_aralik(0, GUN - 1);
        if (gorulen[g] == damga) {
            return 1;
```

```
        }
        gorulen[g] = damga;
    }
    return 0;
}
```

Kesin değer ↔ Monte Carlo tahmini (ortak LCG, tohum 42, satır başına 5000 deneme): n = 10: 0,1169 ↔ 0,1216; n = 23: 0,5073 ↔ 0,5140; n = 40: 0,8912 ↔ 0,8848; n = 60: 0,9941 ↔ 0,9944.

En büyük sapma 0,0137; 5000 denemede tahminin standart sapması en çok √(0,25/5000) ≈ 0,0071 olduğundan beklenen dalgalanma içindedir. **Karma bağlantısı:** 32 bitlik karma değerlerinde ($m = 2^{32}$) %50 eşiği 77 164 anahtardır, ilk çakışmanın beklenen sırası 82 137,9 ($\sqrt{(\pi m/2)} \approx 82\,137{,}2$): 10^5 anahtarda iki 32 bitlik karmanın çakışması neredeyse kesindir; karma yalnız kovayı seçer, eşitlik anahtarla denetlenir.

5.5 Uygulama kalıpları ve önbellekleme (H13'e köprü)

Bölümün dört kalıbı (sayaç: Örnek 7; küme: Örnek 8 ve Alıştırma 10.11; harita: iki toplam; anahtar tasarımı: Örnek 9) kaba kuvvetin $\Theta(n^2)$'sini beklenen $\Theta(n)$'e indirir. Beşinci bir kullanım önbelleklemedir.

Önbellekleme. Pahalı bir fonksiyonun sonucunu girdisiyle bir karma tablosunda saklamak hesabı tekrarlamamayı sağlar (web önbellekleri, sembol tabloları). Özyinelemede buna **memoization** denir ve H13'ün ilk adımıdır: fib(30)'u doğrudan özyinelemeyle hesaplamak 2 692 537 çağrı yapar ($C(n) = C(n-1) + C(n-2) + 1$), n'yi anahtar yapan tablo her değeri bir kez, 31 değer hesaplar. Nedeni **örtüşen alt problemlerdir**.

5.6 Güvenlik: karma taşkını ve savunmalar (Örnek 13)

Karma fonksiyonu **herkesçe bilinen** bir formülse saldırgan tek kovaya düşecek anahtarlar hesaplayabilir. Polinom 31'de (Java'nın `String.hashCode` formülü; burada UTF-8 baytlarıyla) h("Aa") = 65·31 + 97 = 2112 = 66·31 + 66 = h("BB"). Eşit uzunluklu bloklarda h(xy) = h(x)·31^|y| + h(y) olduğundan k bloğun her dizilişi aynı karmayı verir: 2^k çakışan anahtar (karma: Bölüm 3.3'teki polinom; anahtar üretimi):

c/H10_13_karma_saldirisi.c · satır 213–221

```c
static void carpisan_anahtar(size_t i, int k, char *s)
{
    for (int b = 0; b < k; b++) {
        const char *blok = (i >> (k - 1 - b)) & 1u ? "BB" : "Aa";
        s[2 * b] = blok[0];
        s[2 * b + 1] = blok[1];
    }
    s[2 * k] = '\0';
}
```

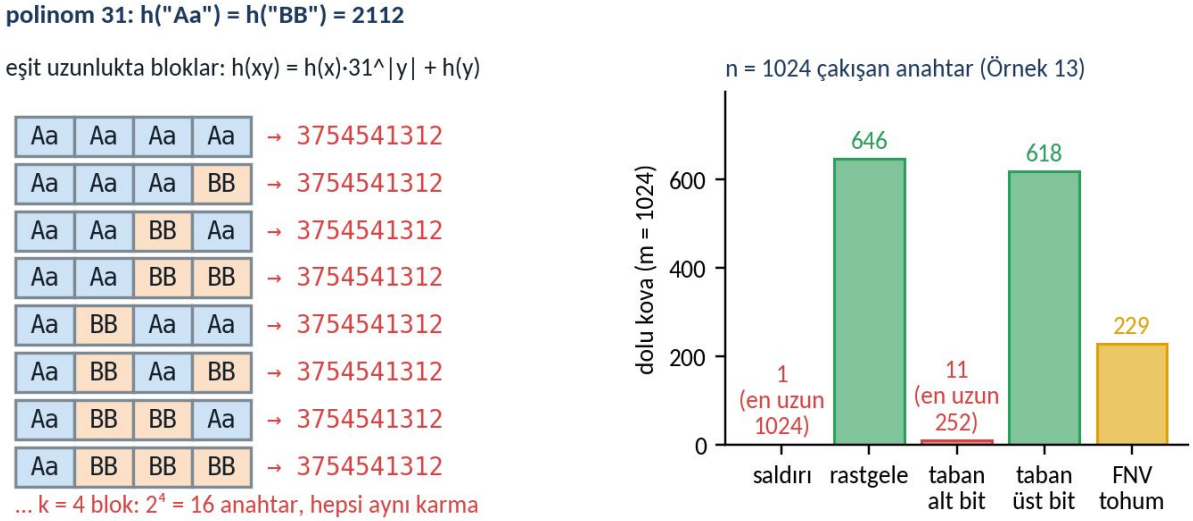
ÇIKTI

```
$ ./H10_13_karma_saldirisi
  h("Aa") = 65·31 + 97 = 2112, h("BB") = 66·31 + 66 = 2112
  AaAaAaBB -> 3754541312   (rastgele taban 211544575 ile:  291934816)
  AaAaBBAa -> 3754541312   (rastgele taban 211544575 ile:    1643104)
  …
```

Saldırıda her ekleme bütün zinciri tarar (anahtar zaten var mı?): karşılaştırma tam n(n − 1)/2, n × 2'de 4,00 kat: **karesel**; rastgele anahtarda n = 1024'te yalnız 782. Örnek 13 dört "tohumlu" savunmayı dener (gizli değerler ortak üreteçten, tohum 2026). Sonuçlar:

- **Tohumlu başlangıç değeri (h_0) hiç savunmaz:** eşit uzunluklu her anahtara aynı $h_0 \cdot 31^{L}$ eklenir, çakışma bozulmaz (n = 1024'te yine 523 776).
- **Rastgele taban + alt bitlerden indis güvenilmez:** karmalar artık eşit değildir, ama indisin en az alt t + s biti ($t = v_2(\text{taban} - 31) \geq 1$, $s = v_2(\text{taban}^2 - 1) \geq 3$) yalnız BB bloklarının **sayısını** taşır (Örnek 13'teki türetme). Örnek 13'ün gizli tabanında t + s = 15: n = 1024'te yalnız 11 kova dolu (en uzun 252), n × 2'de karşılaştırma 3,37 ve 3,69 kat: karesel kalır. Kavram 9 aynı saldırıyı 20 rastgele tabanla dener: n = 1024'te 1 914 ile 93 040 karşılaştırma (ortanca 4 594), 2 tabanda karesele yakın. Savunma tabanın şansına kalır.
- **Rastgele taban + üst bit (çarpma yöntemi) ve tohumlu FNV-1a büyümeyi doğrusallaştırır:** n = 1024'te 618 ve 229 kova dolu (rastgelede 646), karşılaştırma 896 ve 4 133. Bedelsiz değil: FNV-1a bu düzenli anahtarlarda rastgeleden ≈ 5,3 kat fazla karşılaştırır (sabit kat); taban + üst bitte anahtar başına karşılaştırma n ile yavaşça artar (Bölüm 6.4; bu makinede gözlendi, nedeni kanıtlanmadı).
- **Gerçek savunma anahtarlı karmadır:** basit tohumlar formülün yapısını korur, tohum gözlemlerden de çıkarılabilir (Aumasson ve Bernstein Python 3.3'ün tohumunu geri kazandı; Heimes, 2013). SipHash 128 bitlik gizli anahtarlı, 64 bit çıktılı, kısa girdiler için kriptografik bir sözde rastgele fonksiyondur (Aumasson & Bernstein, 2012); Python 3.4'ten beri `str`/`bytes` karmasıdır.

Şekil 10.16
Karma taşkını (Örnek 13): Aa/BB anahtarlarının karması aynı; n = 1024'te dolu kova ve en uzun zincir



Kriptografik karma tablo karması değildir. Tablo karması (FNV-1a, polinom) hız ve düzgün dağılım içindir; çakışma bulmak kolaydır. Kriptografik karma (SHA-256) çakışma bulmayı ve girdiyi geri çıkarmayı hesaplama açısından olanaksız kılar; parola, imza, bütünlük denetiminde kullanılır. Anahtarlı karma (SipHash) arada durur: kısa girdide hızlıdır, gizli anahtar bilinmeden çakışma üretilemez; amacı tam olarak tabloyu saldırgana karşı korumaktır.

6. Haftanın Deneyi: Yük Faktörü, Bellek Erişimi ve Saldırı

Üç soru: (a) α yoklama sayısını ve arama süresini nasıl etkiler, kuramla uyuşur mu? (b) Saldırı altında ekleme nasıl büyür, savunmalar ne kazandırır? (c) Elle yazılmış tablo kütüphaneyle nasıl karşılaştırılır? Yoklama ve karşılaştırma sayıları üç dilde aynıdır (sürücü denetler); süreler bu makineye özgüdür.

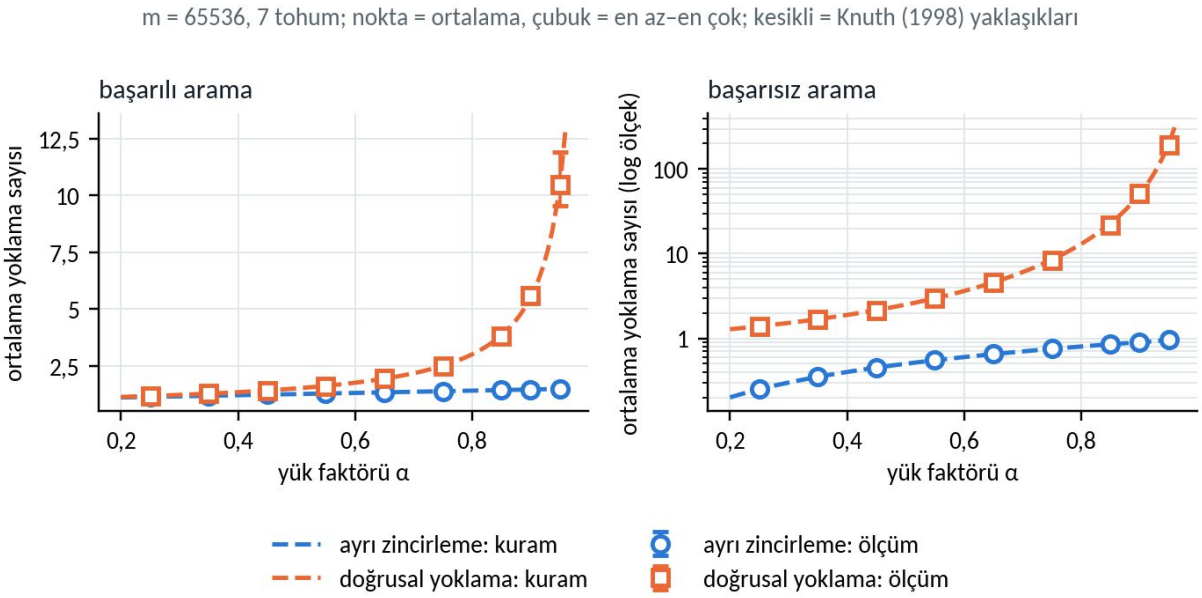
6.1 Ölçüm düzeni

- **Makine ve süre:** Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdek (sanal makine); gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; Python 3.11.15; C/C++ -std=c17 / -std=c++17 -Wall -Wextra -pedantic -O2 (saldırı deneyinde C ayrıca -O0). 1 ısınma + 5 tekrar, ortanca; duvar saati ve işlemci zamanı birlikte kaydedildi; çözümlemede işlemci zamanı kullanıldı (makinede başka işler de vardı).
- **(a)** Yoklama: m = 2^{16}, 7 tohum, α = 0,25 … 0,95, başarılı ve başarısız arama. Süre: m = 2^{20} (8 MB, önbellekten büyük); C/C++ elle iki yapı, `std::unordered_map`, Python `dict` (5 α).
- **(b)** Saldırı: n = 2^{8} … 2^{16}, zincirlemeli tablo (m = 16, α > 1'de × 2); saldırı, rastgele, tohumlu FNV-1a, rastgele taban + üst bit. Python saldırı serisi bütçe nedeniyle 2^{12}'ye dek ölçüldü; üstü $c \cdot n^2$ ile **işaretli kestirim**.
- **(c)** Kelime sayacı: Örnek 7'nin girdisi, elle yazılmış tablo ve Kavram 7, üç dilde. Toplam ölçüm 221,3 s.

6.2 Yük faktörü → ortalama yoklama: kuram ve ölçüm

Ölçüm kuramı izler (Şekil 10.17): ayrı zincirlemede bütün α'larda sapma %1,16'dan küçüktür; bütün tablodaki en büyük mutlak sapma %4,10'dur (doğrusal yoklama, başarısız arama, α → 1). Doğrusal yoklamada başarısız aramanın tohumlar arası göreli yayılımı α ile büyür: α = 0,75'te %3,0, 0,90'da %11,5, 0,95'te %13,2. α = 0,95'te ortalama kuramın (200,4756) %4,1 altındadır, yayılımın içinde. Zincirlemede başarısız arama α'ya **doğrusal**, doğrusal yoklamada $1/(1-\alpha)^2$'ye bağlıdır. Örnek 5 aynı ölçümü tek tohumla m = 2^{13}'te yapar: α = 0,95'te başarısız arama 266,982 yoklama, kuram 200,110; tek bir tablo birkaç uzun kümenin şansına bağlıdır, 7 tohumun ortalaması ise kurama yakındır.

Şekil 10.17
Yük faktörüne karşı ortalama yoklama: kuram (Knuth, 1998) ve ölçüm (m = 2^{16}, 7 tohum)



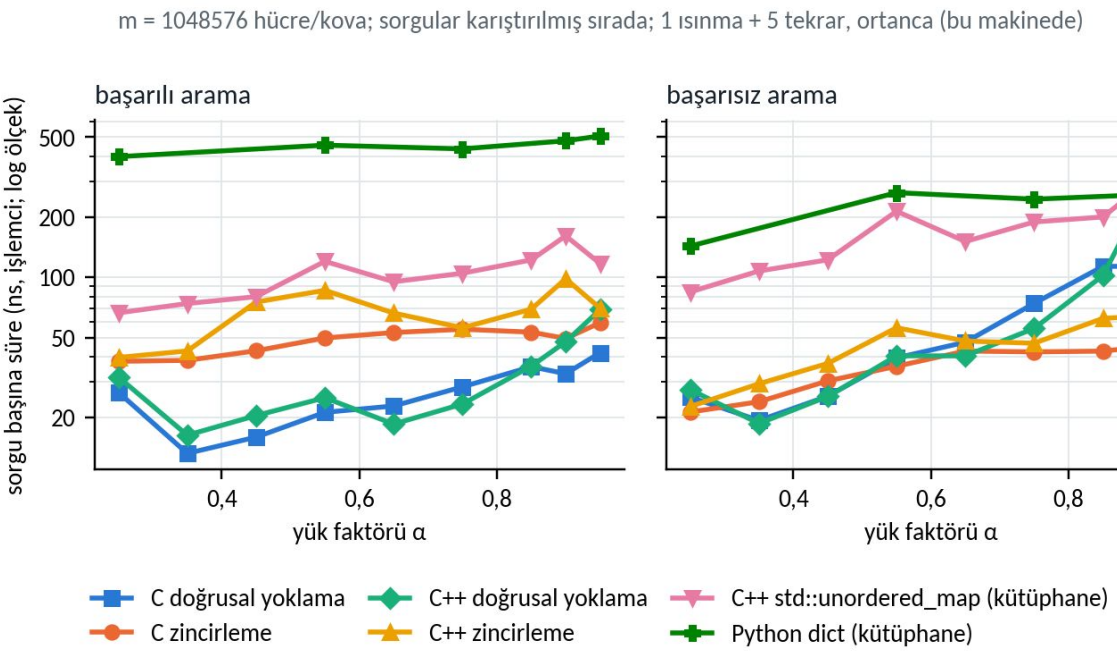
Tablo 10.10
Ortalama yoklama, $m = 2^{16}$, 7 tohumun ortalaması (Z: ayrı zincirleme, D: doğrusal yoklama; kuram: Knuth, 1998)

α	Z başarılı	Z kuram	Z başarısız	Z kuram	D başarılı	D kuram	D başarısız (std)	D kuram
0,25	1,1238	1,1250	0,2505	0,2500	1,1641	1,1667	1,3890 (0,0107)	1,3889
0,55	1,2754	1,2750	0,5564	0,5500	1,6105	1,6111	2,9859 (0,0887)	2,9690
0,75	1,3748	1,3750	0,7531	0,7500	2,4963	2,5000	8,3284 (0,2503)	8,5000
0,90	1,4496	1,4500	0,8930	0,9000	5,5731	5,4997	51,0893 (5,8948)	50,4939
0,95	1,4748	1,4750	0,9552	0,9500	10,4762	10,4994	192,2656 (25,2872)	200,4756

6.3 Aynı tablolarda arama süresi: bitişik hücreler ve dağınık düğümler

Bu makinede başarılı aramada zincirleme, doğrusal yoklamadan α ≤ 0,75 için 1,43–2,91 kat yavaştır, oysa yoklaması daha **azdır** (Tablo 10.10): bitişik hücreler önbellek satırını paylaşır, düğümler dağınıktır (Bölüm 3.5). Başarısız aramada α büyüyünce doğrusal yoklama patlar ve sıra döner: α = 0,85, 0,90 ve 0,95'te zincirleme/doğrusal oranı 0,38, 0,39, 0,16. `unordered_map` düğüm tabanlı, `dict` açık adreslemeli ama yorumlayıcıdadır (α = 0,75 başarılı aramada C doğrusal yoklamadan 15,4 kat yavaş). α'ya göre tekdüze olmayan noktalar makinedeki öbür işlerin gürültüsüdür.

Şekil 10.18
Arama süresi (ns, bu makinede): doğrusal yoklama, zincirleme ve kütüphane tabloları



6.4 Saldırı altında ekleme: karesel büyüme ve savunmalar

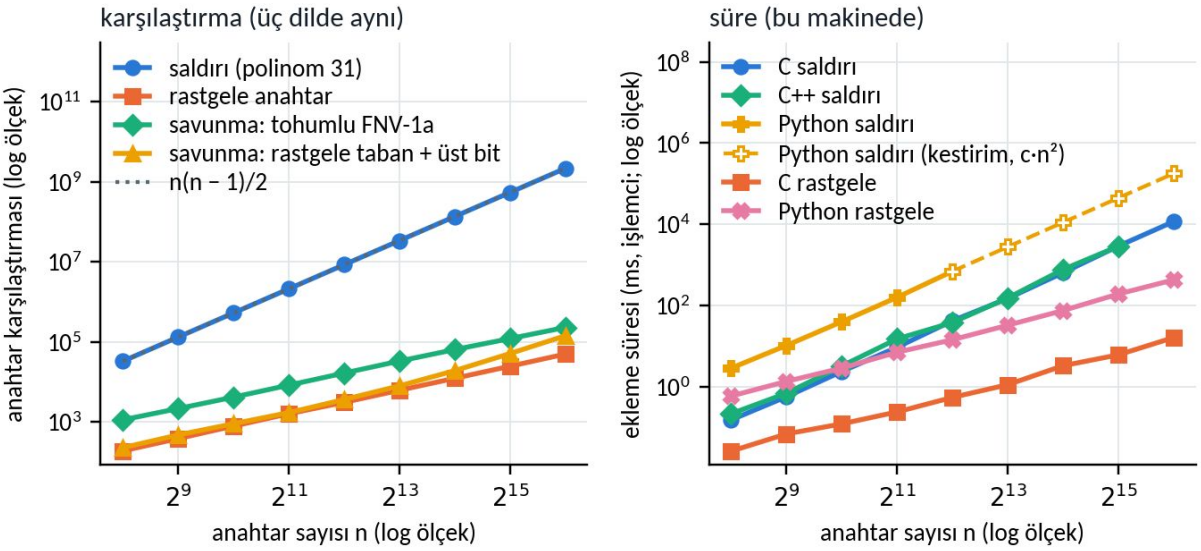
Saldırı serisinde karşılaştırma tam n(n − 1)/2'dir; log–log eğimler bunu süreye de taşır: C 2,04, C++ 1,94, Python 1,98, yani süre ≈ n^2 (≥ 1 ms noktalar). Rastgele anahtarda karşılaştırma eğimi 1,000, süre eğimi 1,25 (büyük n'de önbellek kaçırmaları). n = 2^{16}'da C'de saldırı 718,2 kat yavaştır (Tablo 10.11). Savunmalarda anahtar başına karşılaştırma (n = 2^8, 2^{12}, 2^{16}): rastgele 0,73, 0,75, 0,75; tohumlu FNV-1a 4,38, 4,03, 3,48 (sabit kat, eğim 0,966); rastgele taban + üst bit 0,88, 0,88, 2,15 (eğim 1,141: karesel değil, ama n ile artıyor; bu ölçümde gözlenen bir eğilim, nedeni kanıtlanmadı). Tohumlu h_0 ve taban +

alt bit deneye alınmadı: Örnek 13 bu gizli tabanla karesel kaldıklarını gösterir; alt bitin sonucu tabana bağlıdır (Kavram 9, Bölüm 5.6).

Tablo 10.11
*Saldırı altında n ekleme: karşılaştırma (üç dilde aynı), işlemci zamanı (C -O2, Python; bu makinede); kat = C saldırı/rastgele; * $c{\cdot}n^2$ kestirimi*

n	saldırı kar.	rastgele kar.	FNV tohum kar.	taban/üst kar.	C saldırı	C rastgele	kat	Python saldırı
2^{8}	32 640	188	1 121	226	0,153 ms	26,0 µs	5,9	2,88 ms
2^{12}	8 386 560	3 061	16 504	3 600	42,1 ms	0,547 ms	76,9	695,4 ms
2^{13}	33 550 336	6 134	32 885	7 907	149,1 ms	1,12 ms	132,9	≈ 2,78 s*
2^{16}	2 147 450 880	49 209	227 785	141 094	11,67 s	16,3 ms	718,2	≈ 3,0 dk*

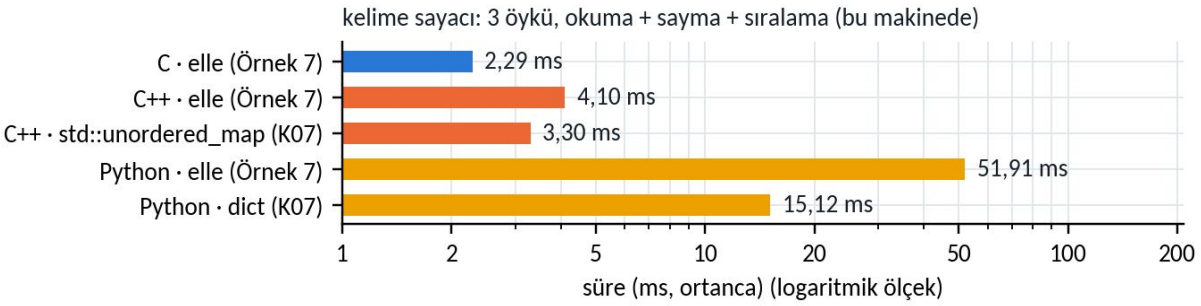
Şekil 10.19
Saldırı altında ekleme (log–log): karşılaştırma ve süre; Python'da 2^{13} üstü işaretli kestirim



6.5 Kelime sayacı: elle yazılmış tablo ve kütüphane

Python'da elle yazılmış tablo `dict`'ten 3,4 kat yavaştır (her yoklama ayrı bayt kodu yürütür; `dict` C'de yazılmıştır); C++'ta elle yazılmış tablonun `std::unordered_map`'e oranı 1,24'tür; en hızlısı C'nin elle yazılmış tablosudur. Üretimde Python'da `dict`/`Counter` kullanılır, C'de standart karşılık olmadığından elle yazmak gerçek bir karardır (Bölüm 8).

Şekil 10.20
Kelime sayacı (Örnek 7, Kavram 7): elle yazılmış tablo ve kütüphane, üç dilde; ortanca süre, bu makinede



6.6 Sınırlılıklar ve kendi makinenizde

- Süreler “bu makinede”dir (Bölüm 6.1), sayaçlar makineden bağımsızdır; m = 2^{20}'lik tablo önbellekten büyüktür, küçük m'de bitişik/dağınık farkı küçülür.
- Kendi makinenizde: `cd kodlar/deney && python3 h10_olcum_calistir.py && python3 h10_cozumle.py` (bu makinede ≈ 3,7 dk). Süreleriniz farklı olacaktır; yoklama ve karşılaştırma sayıları, eğimler ve sıralar aynı kalmalıdır.

7. Hata Avcısı

Hatalı ve düzeltilmiş programlar `kodlar/hata_avcisi/` altındadır; belirtiler bu makinede gözlenen çıktılardır. Hepsi uyarısız derlenir. Hata Avcısı 1 tanımsız davranıştır: belirtisi “bu derleyicide, bu çalıştırmada” geçerlidir.

Tablo 10.12
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Negatif karma, `%` ile negatif indis	`karma = -57261580, indis = h % 11 = -2 şule`, ardından `munmap_chunk(): invalid pointer` (SIGABRT)	İşaretli `char` baytı (0xC5 → negatif) ve `int` taşması negatif karma üretir; C'de `%` bölünenin işaretini alır (H1)	`uint32_t` karma, `unsigned char` bayt: indis her zaman 0 … M − 1
2	Anahtar dizgisi kopyalanmıyor	bütün kovalarda `deniz`; `ara ali: YOK`	Düğüm, okuma tamponunun adresini saklıyor; tampon değişince bütün anahtarlar değişir	Düğüm anahtarın kendi kopyasını tutar (`malloc` + `memcpy`) ve onunla serbest bırakılır
3	Silinen hücreyi BOŞ yapmak	`ara ece: YOK — tabloda olduğu hâlde! (3 yoklama)`; yeniden eklenince `tablodaki "ece" sayısı: 2 (1 olmalı)`	Arama BOŞ hücrede durur; silinen hücrenin ötesine yerleşmiş anahtarlar kaybolur	Mezar taşı: aramayı durdurmaz, eklemede yeniden kullanılır
4	Yük denetimi yok (tablo doluyor)	`ekle ılgın: 1000000 yoklamadan sonra DURDURULDU — BOŞ hücre yok`	α = 1'de BOŞ hücre kalmaz; döngü yalnız BOŞ hücrede biter (gösterimde adım sınırı var)	Eklemeden önce (n + 1)/m > 3/4 ise × 2; yoklama en çok m hücre
5	Rabin–Karp'ta doğrulama yok	`bildirilen eşleşme: 7578 (doğrusu 1)`	Karma eşitliği yalnız adaydır; q = 13'te ≈ her 13 pencereden biri sahte	Adayı `memcmp` ile doğrula; sahteleri say
6	Python'da `list` anahtarı	`TypeError: unhashable type: 'list'`	`list` değiştirilebilir, karma edilemez	`tuple(sorted(s))`, `"".join(sorted(s))` ya da `bytes`

Hata Avcısı 1'e yakından bakış. `karma_kotu` (`int h, h = 31 * h + *s;`) baytı işaretli `char` olarak okur (“ş”nin ilk baytı 0xC5 bu derleyicide −59), uzun anahtarda `int`'i taşırır; negatif karmanın `int j = h % M;` kalanı negatiftir ve `sayac[j]++` (41. satır) dizinin **önüne** yazar.

AddressSanitizer dizinin 8 bayt önüne erişimi (`sayac[-2]`), UndefinedBehaviorSanitizer işaretli taşmayı bildirir (adres, süreç numarası ve yol sadeleştirildi):

ASAN / UBSAN (SADELEŞTİRİLMİŞ)

```
$ gcc -std=c17 -O1 -g -fsanitize=address,undefined H10_HA1_negatif_indis.c && ./a.out
==PID==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x…
READ of size 4 at 0x… thread T0
#0 0x… in main /…/kodlar/hata_avcisi/H10_HA1_negatif_indis.c:41
0x… is located 8 bytes before 44-byte region [0x…,0x…)
SUMMARY: AddressSanitizer: heap-buffer-overflow
/…/kodlar/hata_avcisi/H10_HA1_negatif_indis.c:41 in main

/…/kodlar/hata_avcisi/H10_HA1_negatif_indis.c:22:16: runtime error: signed integer
overflow: -1825742406 * 31 cannot be represented in type 'int'
```

Düzeltilmiş sürümde karma `uint32_t`, bayt `unsigned char`'dır: `karma = 186786292, indis = h % 11 = 0 şule`. Mini-OBS 2 v2.9 indisi işaretsiz çarpmanın üst bitlerinden alarak bu tuzaktan kaçınır (v2.9 belgesi § 10).

Hata Avcısı 2 ve 3 çökmez, sessizce yanlış sonuç verir: düğüm yalnız işaretçiyi saklar (`d->anahtar = anahtar;`) ya da silinen hücre BOŞ yapılır (`hucre[i] = NULL;`); mezar taşlı düzeltme `ara ece: bulundu (7 yoklama)` der (Şekil 10.10).

Hata Avcısı 4 ve 5. Yük denetimsiz tablo dolunca on ikinci eklemede ve olmayan anahtarın aramasında sonsuz döngüye girer (gösterimde 10^6 yoklamalık sınır var; Tablo 10.12). Düzeltilmiş sürüm dokuzuncu eklemeden önce büyür (9/11 > 3/4: `(büyütme: m 11 -> 22)`). Doğrulamasız Rabin–Karp her karma eşitliğini eşleşme sayar:

C hata_avcisi/H10_HA5_dogrulamasiz_rabin_karp.c · satır 36–38

```c
        if (hp == ht) {                     /* HATA: doğrulama yok */
            if (say < 5) {
                printf(" %zu", s);
```

Cormen vd. (2022) örneğinde `bildirilen eşleşme: 2 (doğrusu 1: yalnız kayma 6; 12'deki 67399 sahte)`; düzeltilmiş sürüm rastgele metinde `konumlar: 50000; sahte eşleşme (elenen) = 7577` bildirir. **Hata Avcısı 6** Python'a özgüdür (Bölüm 5.3); düzeltilmiş anahtar `aikpt: kitap takip` grubunu verir.

8. Dil Penceresi: C · C++ · Python

Tablo 10.13
Dil karşılıkları: karma tabloları, karma edilebilirlik, dolaşma sırası ve bayt işlemleri

Öğe	C	C++	Python
Genel amaçlı karma tablosu	standartta yok; elle ya da POSIX `hsearch` (Kavram 6)	`std::unordered_map<K, V>`	`dict`
Karma kümesi	elle (Örnek 8)	`std::unordered_set<K>`	`set`, `frozenset`
Yükü ve büyümeyi kim yönetir?	programcı (Örnek 6)	`max_load_factor()` = 1,0 başlangıçta; `rehash`, `reserve`	yorumlayıcı (CPython: 2/3)

Öğe	C	C++	Python
Dolaşma sırası	programcının yapısı	belirtilmemiş; yeniden karmada değişebilir	dict: ekleme sırası (3.7+); set: güvence yok
Dizgi karması	elle (FNV-1a, polinom)	std::hash<std::string>: gerçekleştirime bağlı	hash(): SipHash, süreç başına rastgele tohum
Kullanıcı türü anahtar	karma + eşitlik fonksiyonu elle	operator== + std::hash<T> özelleştirmesi	__eq__ + __hash__; anahtar değişmez olmalı
Küçük harf	tolower: bayt bayt, yerel ayara bağlı	std::tolower (aynı)	str.lower: Unicode kuralları (İ → i)
Negatif sayının kalanı	bölünenin işareti: -7 % 2 = −1	aynı	bölenin işareti: -7 % 2 = 1
Tamsayı taşması	işaretsiz: mod 2^w sarar; işaretli: tanımsız	aynı	taşma yok; & 0xFFFFFFFF ile 32 bit

C: standart karma tablosu yoktur. POSIX <search.h> hcreate/hsearch/hdestroy sunar: süreçte tek tablo, silme ve dolaşma yok, kapasite baştan verilir, anahtarın ömrünü çağıran yönetir. Kavram 6 küçük bir kelime sayacını hsearch ile yazar: 5 farklı sözcük sayılır, ama dolaşma işlemi olmadığından anahtar listesini program ayrıca tutmalıdır. C'de tablo çoğu zaman elle yazılır.

Dolaşma sırası. Python 3.7'den beri dict'in ekleme sırasıyla dolaşması dil güvencesidir (Python Software Foundation, t.y.-d); güncelleme sırayı değiştirmez, silip eklemek sona taşır; set için güvence yoktur. std::unordered_map'in sırası belirtilmemiştir: Kavram 4'te "rize" silinip yeniden eklenince dict artvin, trabzon, giresun, ordu, rize sırasını verir; unordered_map aynı anahtarları 13 kovada ordu, giresun, trabzon, artvin, rize sırasıyla dolaşır ve rehash sonrası sıra değişir. Bu yüzden örnekler anahtarları yazdırmadan önce **sıralar** (çıktı eşdeğerliği). CPython dict'i açık adreslemelidir: silinen yuva mezar taşıdır (DKIX_DUMMY; kaynak koddaki açıklamaya göre boş yapılırsa yoklama dizisi yuvanın dolu olduğunu bilemez) ve kapasitenin 2/3'ü kullanılır (USABLE_FRACTION; Python Software Foundation, t.y.-a): Bölüm 4.2'nin mezar taşı ve yük eşiği üretimde aynen vardır.

Karma rastgeleleştirmesi. Python'un hash()'i str ve bytes için her süreçte rastgele seçilen gizli bir anahtarla hesaplanır: Kavram 3'ün iki çalıştırmasında hash('ali') -6495032268520799091 ve 1721433872238283405 çıktı; PYTHONHASHSEED=0 rastgeleliği kapatır ve değer sabitlenir (-1980403479539151055). Bu yalnız hata ayıklama içindir; üretimde kapatmak Bölüm 5.6'daki saldırıya kapı açar. hash(42) = 42; libstdc++'ta std::hash<std::string> çalıştırmalar arasında aynıdır (tohum yok). Ders örneklerinin çıktısında ikisi de kullanılmaz.

Karma edilebilirlik sözleşmesi. a == b ise hash(a) == hash(b) olmalı ve anahtarın karması tablodayken değişmemelidir. Python'da yalnız __eq__ tanımlayan sınıfın __hash__'i None olur; ikisi aynı alandan tanımlanır, alan değişirse kayıt eski kovada "kaybolur". C++'ta operator== ve std::hash<T> özelleştirmesi gerekir; düğümdeki anahtar const'tur (Kavram 5). Python: dict anahtarı yapılamadı: TypeError: unhashable type: 'OgrenciEq';a in notlar: False (kayıt tabloda ama eski karmanın kovasında). C++: a == b: true, karmalar eşit: true, notlar[b] = 85.

Büyük/küçük harf. C'nin `tolower`'ı bayt bayt ve yerel ayara bağlı çalışır: "C" yerel ayarında yalnız A–Z değişir, iki baytlık Ç, Ğ, İ, Ş kalır, "I" Türkçedeki "ı" yerine "i" olur. Python'un `str.lower`'ı Unicode kurallarıyla Ş → ş yapar, ama I → i ve İ → i + U+0307 (birleşen üst nokta) olur (Kavram 8):

KAVRAM 8 · C VE PYTHON

```
$ ./H10_K08_kucuk_harf
IŞIK: tolower -> iŞik, elle -> iŞik, aynı mı? evet

$ python3 H10_K08_kucuk_harf.py
IŞIK: lower -> işik, elle -> iŞik, aynı mı? hayır
İSTANBUL: lower -> istanbul, elle -> İstanbul, aynı mı? hayır
   8 → 9 karakter: U+0069 U+0307 U+0073 U+0074 U+0061 U+006E U+0062 U+0075 U+006C
```

İkisi de Türkçe için doğru değildir ve üç dilde aynı sonucu vermez; Örnek 7 bu yüzden bayt düzeyinde **elle** ASCII dönüşümü yapar (A–Z → +32): sonuç yerel ayardan bağımsız ve üç dilde aynıdır. Türkçe kurallı katlama (I ↔ ı, İ ↔ i) Mini-OBS 2'de kendi tablosuyla yapılır (v2.9 belgesi § 10). **Kalanın işareti** (Bölüm 4.5, Hata Avcısı 1) işaretsiz tür ve "önce + q" kuralıyla etkisizleşir.

9. Mühendis Gözüyle: Ortalama Durum Bir Varsayımdır

- **"O(1)" bir sözleşme değil, bir varsayımın sonucudur.** Anahtarları dışarıdan biri seçiyorsa (HTTP parametreleri, JSON alanları, ağ paketleri) varsayım saldırganın eline geçer: n = 2^{16}'da saldırı altında ekleme rastgele anahtardan 718,2 kat yavaştır (Bölüm 6.4). Dışarıdan gelen anahtar için **rastgele tohumlu, tercihen anahtarlı** karma (SipHash) kullanın.
- **Savunmayı ölçün, varsaymayın.** "Tohum ekledim" Örnek 13'te iki kez yetmedi: h_0 tohumu hiç savunmaz, alt bit indisi gizli tabanın şansına kalır (Kavram 9: 20 tabandan 2'sinde karesele yakın). Savunma saldırının kullandığı yapıyı bozuyor mu, sayaçla ve birden çok tohumla denetleyin.
- **Katmanlı korunma.** Sorun POST parametre sayısının sınırsızlığıyla birleşince ortaya çıktı (oCERT, 2011): girdi boyutunu sınırlamak ikinci katmandır.
- **Yük eşiği bir bellek–zaman ödünleşimidir** (trade-off). Kurama göre doğrusal yoklama başarısız aramada α = 0,75'te 8,5, α = 0,95'te 200,5 yoklama yapar (Bölüm 6.2). Bellek darsa zincirleme ya da akıllı yoklama (Robin Hood, geri kaydırmalı silme); hız kritikse düşük α ve bitişik hücreler.
- **Test ve ölçüm:** mezar taşı, büyütme ve silme hataları çoğu girdide görünmez: rastgele ekle/ara/sil dizilerini bir modelle karşılaştırıp değişmezi denetleyin (Alıştırma 10.12); yoklama sayısını kuramla, süreyi bellek hiyerarşisiyle birlikte okuyun (Bölüm 6.3).
- **Ne zaman karma tablosu değil?** Sıralı dolaşma, aralık sorgusu, en küçük/en büyük anahtar ya da en kötü durum güvencesi gerekiyorsa sıralı yapı (H4, dengeli ağaçlar); evren küçükse doğrudan adresleme.

10. Mini-OBS 2 v2.9: Numara Dizini, O(n) Yinelenen Tespiti ve Alt Dizgi Araması

Bu hafta: Mini-OBS'ye bir **numara → kayıt karma tablosu** ekliyoruz (açık adresleme, doğrusal yoklama, mezar taşı, yük eşiği ve × 2 yeniden boyutlandırma), yinelenen numara tespitini **karma kümesiyle O(n)'e** indirip v2.1 ($\Theta(n^2)$) ve v2.5 ($\Theta(n \log n)$) ile aynı veride karşılaştırıyoruz ve ad–soyad içinde **büyük/küçük harf duyarsız alt dizgi araması** için naif yöntemi ve **Rabin–Karp**'ı yazıyoruz. (v2.9 belgesi § 10)

10.1 Gereksinimler

- `karma.h/.c`: `long long` anahtar → `int` değer tablosu; açık adresleme + doğrusal yoklama; kapasite 2'nin kuvveti; Fibonacci karması `ev(k) = (A·k mod 2⁶⁴) >> (64 − p)`; hücre durumu BOŞ/DOLU/MEZAR; `karma_olustur`, `karma_bul`, `karma_ekle`, `karma_sil` (mezar taşı), `karma_yok_et`; yük (n + mezar)/kapasite > 0,7 olacaksa × 2 yeniden boyutlandırma; değişmez denetleyicisi.
- `karma_dizin_kur(t, sinif)`: numara → kayıt indisi; `ara --no N --karma` yoklama sayısını basar.
- `yinelenen_bul_karma`: v2.1 ile **aynı çiftler, aynı sözlük sırası**; `denetle --yontem karma`.
- `desen.h/.c`: katlanmış ad–soyad metni; `desen_ara` naif ve Rabin–Karp (d = 256, q asal; **karma eşitliği bayt bayt doğrulanır**; aday ve sahte eşleşme sayaçları); `ara --icinde DESEN` üç yöntemin (naif, Rabin–Karp, `strstr`) sonucunu karşılaştırır.
- Türkçe katlama: v2.4'ün sıra değeriyle uyumlu (`tr_katla`): "yıl" → "YIL", "ilgaz" → "İLGAZ", "âşık" → "AŞIK".
- Testler: karma tablosu ↔ düz model (rastgele işlem dizileri, her adımda değişmez); karma kümesi ↔ v2.1; desen ↔ harf düzeyinde kâhin ve `strstr`.
- Ölçüm: ikili arama ↔ karma (10^5–10^6); üçlü yinelenen n = 10^3…10^6 ($\Theta(n^2)$ için süre bütçesi + işaretli kestirim); yük faktörüne göre ortalama yoklama ve Knuth'un formülleri; 10^5 kayıtta alt dizgi araması (sahte eşleşme sayısıyla).

10.2 İpuçları

- Kapasiteyi 2'nin kuvveti tutun: indis `(carpan * (uint64_t)k) >> (64 - p)`; `%` yok, negatif indis yok (H1'in kalan işareti tuzağı burada çıkmaz). İşaretli taşma tanımsızdır, **işaretsiz** taşma mod 2^{64} sarar: çarpımı `uint64_t` ile yapın.
- Silmede hücreyi BOŞ yapmayın; MEZAR yapın. Aramada MEZAR'ın üzerinden geçin, eklemede ilk MEZAR'ı hatırlayın ama yokluğu BOŞ hücreye kadar doğrulayın.
- Eşiği (n + mezar) ile denetleyin; mezarlar da yoklamayı uzatır. Yeniden kurmada yalnız DOLU hücreleri taşıyın.
- Tabloyu `calloc` ile ayırın: hepsi BOŞ (durum 0) başlar. Yeniden kurmada yeni diziyi ayıramazsanız eskisini **bırakmayın** (tablo değişmesin).
- Yinelenen tespitinde değer olarak grubun **ilk** indisini, ayrıca her grubun sonunu ve boyunu tutun: çiftler sözlük sırasında yazılabilir, toplam Σ C(g, 2) yazmadan bulunur.
- Rabin–Karp'ta karma eşitliğinden sonra **mutlaka** baytları karşılaştırın; sayaçta aday = eşleşme + sahte olmalıdır. Küçük bir q (13, 101) ile sahte eşleşmeleri görün.
- Kaydırma formülünde çıkarmadan önce q ekleyin (`(k + q − x) % q`) ya da çıkan baytın payını önceden tablolayın; `uint64_t` ile d·k taşmaz.
- Katlamayı `toupper` ile değil, v2.4'ün tablosuyla yapın; desen ile metni **aynı** fonksiyonla katlayın.

10.3 Başlangıç noktası

- **Verilir:** kendi v2.8'iniz (ya da referans v2.8), `karma.h` ve `desen.h` (bildirimler, sözleşmeler, sahiplik), `tr_katla`'nın tablosu (v2.4'ün `turkce.c`'si), `olcer_karma`'nın ölçüm düzeneği (veri üretimi, sorgu dizisi, saldırı numaralarının üretimi), Şekil 1 ve 2'nin örnekleri.
- **Yazılır:** `karma.c` (bul, ekle, sil, yeniden kurma, değişmez), `yinelenen_bul_karma`, `desen.c` (naif, Rabin–Karp, ad–soyad metni), `ara` ve `denetle`'ye yeni seçenekler; testler (birim + kâhin); ölçüm ve yorum.

- **Değiştirilmez:** v1.0 kuralları ve menü, kayıt dizisinin numara sırası, v2.4'ün Türkçe sırası, ortak modüller.
- **Meydan okuma:** (a) ayrı zincirlemeli tablo yazın ve § 8.4'ün deneyini zincirlemede de yapın (beklenen 1 + α/2 ve α + e^(−α)); (b) robin hood yoklaması ya da geri kaydırmalı silme (mezarsız) ile karşılaştırın; (c) çok desenli Rabin–Karp (k desen, karmalar bir karma kümesinde); (d) KMP'yi yazın ve naifin en kötü durumunda üç yöntemi karşılaştırın.

10.4 Kabul ölçütleri ve test senaryoları

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri (389) ve `test_kahin` (300 deneme) geçer.
2. `./ara ornek_veri/ogrenciler.csv --no 2301010050 --karma` şu satırı basar: `Karma tablosu (doğrusal yoklama) : sıra 6 · 1 yoklama (ev hücresi 5)`; ikili arama aynı kaydı (sıra 6) bulur.
3. 8 hücrelik tabloya (Fibonacci çarpanı) 2301010045, …050, …058 ve …042 eklenince hücreler 2–5 dolar; …050 silinince …058 3 yoklamada bulunur; …071 eklenince 5 yoklamayla [3] hücresine (mezara) yazılır. Kapasite 8'de 6. anahtar tabloyu 16'ya büyütür.
4. `./ara ornek_veri/ogrenciler.csv --icinde yıl` üç yöntemde de `2 kayıt · 2 eşleşme` ve `Üç yöntem: AYNI` basar; `--icinde ilgaz` 0 kayıt, `--icinde ılgaz` 1 kayıt (ILGAZ ŞAHİN) bulur.
5. `./ara ornek_veri/ogrenciler.csv --icinde a --q 13` Rabin–Karp için `Rabin–Karp : 7 kayıt · 12 eşleşme · 117 pencere · 23 aday · 11 sahte · 23 karakter karşılaştırması` basar (sahte eşleşmeler doğrulamayla elenir; eşleşme sayısı naifle aynı).
6. `./denetle testler/yinelenen_k3.csv --yontem karma` v2.1 ve v2.5 ile aynı üç çifti aynı sırada yazar ve 1 ile çıkar; `--erken` aynı ilk çifti (sıra 8 ↔ 11) verir.
7. `./test_kahin --hata-goster mezar` ve `--hata-goster rk` uyuşmazlık basar ve 1 ile çıkar; `make asan` ve `make valgrind` temizdir.
8. Ölçüm raporunda: üç yöntemin n = 10^3…10^6 süreleri (Θ(n^2) için işaretli kestirimle) ve eşit çift sayılarının aynılığı; 10^5–10^6 kayıtta ikili arama ↔ karma; en az beş yük değerinde ölçülen ortalama yoklama ile Knuth'un formülü; 10^5 kayıtta alt dizgi araması süresi ve sahte eşleşme sayısı.

Tablo 10.14
Mini-OBS 2 v2.9 test senaryoları (v2.9 belgesi § 10)

#	Girdi	Beklenen çıktı / davranış
1	8 hücre, anahtarlar 1..8'in evleri (Fibonacci)	[4] [1] [6] [3] [0] [5] [2] [7] (hepsi farklı)
2	ekle 10 → ekle 10 → bul 10	EKLENDİ, ZATEN VAR (değer değişmez), bulundu
3	çarpan 1: ekle 1, 2, 3; sil 2; bul 3	bulundu, 3 yoklama (mezarın üzerinden)
4	aynı tablo: bul 5 (yok)	yok, 4 yoklama ([3] BOŞ)
5	aynı tablo: ekle 4	[1] hücresine (mezar) yazılır; mezar 0
6	kapasite 8, 6 anahtar	6.'da kapasite 16, yeniden kurma 1, taşınan 5
7	büyüme kapalı (eşik %100), kapasite 8	7 anahtar girer, 8.'si reddedilir; arama sonlanır
8	numaralar [5, 7, 7, 5]	çiftler (0, 3), (1, 2); ilk çift (0, 3)

#	Girdi	Beklenen çıktı / davranış
9	[9, 9, 9], azami 2	dönüş 3, yazılan (0, 1), (0, 2)
10	`--icinde ilgaz` / `--icinde ılgaz`	0 kayıt / ILGAZ ŞAHİN
11	“CAN YILMAZ” içinde “YIL”, q = 29	2 aday, 1 sahte, eşleşme konum 4
12	“AAAAAAAA” içinde “AAB”	naif 6 pencere × 3 = 18 karşılaştırma; Rabin–Karp 0 aday

10.5 Üçlü karşılaştırma: v2.1 → v2.5 → v2.9

Tablo 10.15
*Yinelenen numara tespiti, üç sürüm aynı veride (Mini-OBS 2 v2.9 belgesi § 8.3, Tablo 3 ve 4; sıralama ve karmada üç sürecin ortancalarının ortancası). * ölçülmedi: T(n) ≈ T(10⁵)·(n/10⁵)² kestirimi*

n kayıt	v2.1 tüm çiftler Θ(n²)	v2.5 sırala + komşu Θ(n log n)	v2.9 karma kümesi Θ(n) beklenen	çiftler / karma	sıralama / karma	v2.9 yoklama
1 000	166,7 µs	158,5 µs	10,8 µs	15	14,7	1 001
10 000	17,8 ms	1,6 ms	135,1 µs	132	12,1	10 000
100 000	1,89 s	22,9 ms	8,1 ms	232	2,8	100 000
200 000	≈ 7,56 s*	46,6 ms	24,1 ms	≈ 314*	1,9	200 000
1 000 000	≈ 188,96 s*	297,2 ms	160,2 ms	≈ 1 180*	1,9	1 000 000

Sarmalın üç basamağı aynı çiftleri aynı sırada verir (kâhin): H2 (v2.1) bütün çiftler, H6 (v2.5) birleştirmeli sıralama + komşular, H10 (v2.9) karma kümesi. n × 10'da tüm çiftlerin karşılaştırması ≈ 100 kat, karma yoklaması 10 kat artar. Süreler oranlardan sapar: karma sıralamadan n = 10^3'te 14,7, n = 10^6'da yalnız 1,9 kat hızlıdır; v2.9 belgesi (§ 8.3) olası açıklama olarak tablonun L2 önbelleğinden taşmasını gösterir (rastgele erişim ↔ ardışık sıralama). Θ(n²) her ölçekte kaybeder (log–log grafik: sunum ve web sayfası).

11. Alıştırmalar ve Atölyeler

Türler: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez, D) hata bulma, E) Parsons (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Kodlu alıştırmaların dosyaları `kodlar/alistirmalar/` altındadır: önce **çalıştırmadan** tahmin edin. Yazdığınız kodu bir kâhinle karşılaştırın, C/C++'ı AddressSanitizer ile çalıştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 10.1 ★☆☆ *Çıktı tahmini*

M = 7 hücreli doğrusal yoklamalı tabloya ev k % 7 ile 10, 17, 3, 24, 31 eklenir; 17 silinir (MEZAR), 24 aranır, 38 eklenir ve tablo yazılır. Programın `yer_bul`'u (`alistirmalar/H10_A01_cikti_tahmini.c` · satır 17–39) Bölüm 4.2'deki BUL sözde kodunun tamsayı anahtarlı karşılığıdır: k varsa hücresi, yoksa ilk mezar taşı ya da BOŞ hücre. Her satırın çıktısını (hücre, yoklama) ve son tabloyu yazın. 38 neden o hücreye yazılır?

Alıştırma 10.2 ★☆☆ *İzleme*

m = 11 kovalı ayrı zincirlemeli tabloya h(k) = k mod 11 ile, **başa ekleyerek** 22, 1, 13, 11, 24, 33, 18, 42, 31 ekleyin; kovaları çizin. En uzun zincir hangisi; her anahtar bir kez aranırsa başarılı aramada ortalama kaç karşılaştırma yapılır? Aynı anahtarları m = 11 hücreli **doğrusal yoklamalı** tabloya ekleyip son tabloyu ve toplam yoklamayı yazın.

Alıştırma 10.3 ★★☆ *İzleme*

T = 2718281828459045, P = 828, d = 10, q = 11 için Rabin–Karp'ı elle izleyin: h, p ve her s için kayan güncellemeyle t_s (Bölüm 4.5'teki biçimde). Hangi kaymalarda karma eşittir; kaçı gerçek, kaçı sahte eşleşmedir? Naif yöntem kaç karakter karşılaştırır?

11.2 Karmaşıklık (B)

Alıştırma 10.4 ★★☆ *Karmaşıklık*

Başarısız aramada en çok 4 yoklama isteniyor: doğrusal yoklamada α en çok kaç (½(1 + 1/(1 − α)²) ≤ 4), zincirlemede kaç? n = 10^6 anahtar (8 bayt) için bellek: zincirleme (kova 8 + düğüm 16 bayt) ↔ açık adresleme (hücre 8 + 1 bayt)?

Alıştırma 10.5 ★★☆ *Karmaşıklık*

Örnek 6'nın tablosu (m = 8, eşik 3/4) (a) m × 2, (b) m + 1024 ile büyüyor. n = 10^6 eklemede büyütme sayısı, toplam ve anahtar başına yeniden yerleştirme? Hangisi amortize O(1)'dir?

11.3 Değişmez yazma (C)

Alıştırma 10.6 ★★☆ *Değişmez yaz*

Örnek 3'teki `tablo_sil` (`c/H10_03_ayri_zincirleme.c · satır 128–145`) için döngünün her adımının başında `*bag`'in neyi gösterdiğini bir değişmez olarak yazın. Değişmezden (a) baştaki düğümün özel durum gerektirmediğini, (b) döngü `*bag == NULL` ile bitince anahtarın kovada olmadığını çıkarın.

11.4 Hata bulma (D)

Alıştırma 10.7 ★★☆ *Hata bul*

Tablo m = 4'ten başlar; ev hücresi `k & (m − 1)`, doluysa `(i + 1) & (m − 1)`; eşik 3/4 (dosyanın 18–49. satırları). Program 6, 10, 14, 7'yi ekleyip hepsini arıyor; bazı aramalar YOK diyor. Hatalı yeri bulun. Düzeltmeden önce tahmin edin: büyütmeden sonra hangi anahtarlar “kaybolur”, neden? Düzeltmenin büyütme maliyeti nedir?

alistirmalar/H10_A07_hata_bul_buyutme.c · satır 27–35

```
27  static void buyut(void)
28  {
29      int eski_m = m;
30      m = 2 * m;
```

```
    for (int i = eski_m; i < m; i++) {   /* yeni yarı boş başlar */
        dolu[i] = 0;
    }
    printf("büyütme: m %d -> %d\n", eski_m, m);
}
```

Alıştırma 10.8 ★★☆ *Hata bul*

İki toplam (doğrudan adreslemeyle, değerler 0 … 99) a = {3, 8, 5, 1, 9} için T = 9, 6, 16: bir çıktı doğru, ikisi yanlış. Hatalı satırları ve ortak nedeni bulun; Örnek 8'deki sıra (önce ara, sonra ekle) bunu nasıl önler?

alistirmalar/H10_A08_hata_bul_iki_toplam.c · satır 17–27

```
    for (int j = 0; j < n; j++) {
        if (konum[a[j]] == 0) {
            konum[a[j]] = j + 1;
        }
        int tumleyen = T - a[j];
        if (tumleyen >= 0 && tumleyen < EVREN && konum[tumleyen] != 0) {
            *bi = konum[tumleyen] - 1;
            *bj = j;
            return 1;
        }
    }
```

11.5 Parsons (E)

Alıştırma 10.9 ★★☆ *Parsons*

Ayrı zincirlemede `Dugum **` ile silmenin karıştırılmış satırları; iki satır fazladır (dosya bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; fazla satırlar neden yanlıştır? Baştaki, ortadaki ve sondaki düğüm silinebilmelidir.

alistirmalar/H10_A09_parsons_zincir_sil.c · satır 7–23

```
        bag = &(*bag)->sonraki;
    return 0;
            free(silinen);
static int tablo_sil(Tablo *t, const char *anahtar)
            *bag = silinen->sonraki;
        bag = (*bag)->sonraki;
    while (*bag != NULL) {
            Dugum *silinen = *bag;
            return 1;
        if (strcmp((*bag)->anahtar, anahtar) == 0) {
    Dugum **bag = &t->kova[kova_no(t, anahtar)];
            free(silinen->anahtar);
        }
            free(*bag);
    }
{
}
```

11.6 Kod yazma (F)

Alıştırma 10.10 ★☆☆ *Kod yaz*

Bir dizgide **ilk tekrar eden baytı** 256 girdilik "görüldü" dizisiyle tek geçişte bulun; "karma", "abc", "kılıç", "anahtar", boş dizgi; kâhin: iç içe döngü. "kılıç"taki tekrar neden bayttır? C++ ve Python sürümleri de.

Alıştırma 10.11 ★★☆ *Kod yaz*

İki dizinin **kesişimini** (A'daki ilk görülme sırasıyla) açık adreslemeli karma kümesiyle beklenen O(|A| + |B|) sürede bulun (BOŞ / VAR / YAZILDI). Ortak üreteç (tohum 42), |A| = 1000, |B| = 1500, değerler 2 400 000 000 + [0, 1999]: kesişim boyu, yoklama ve kaba kuvvetin karşılaştırması?

Alıştırma 10.12 ★★★ *Kod yaz*

Mezar taşı kullanmadan **geri kaydırmalı silme** (Knuth, 1998, Algoritma 6.4R): boşalan hücreden sonraki kümede ev hücresi boşlukla kendi hücresi arasında (dairesel) olmayan anahtarı boşluğa taşıyın, boşluğu kaydırın, BOŞ hücrede durun. (a) m = 8, ev(1) = ev(9) = ev(17) = 4, ev(6) = 5: 9 silinince tablo? (b) 100 000 rastgele işlemi (tohum 42) bir modelle karşılaştıran, değişmezi denetleyen bir test yazın.

11.7 Diller arası okuma (G)

Alıştırma 10.13 ★★☆ *Diller arası okuma*

C programı ve Python eşi (`alistirmalar/H10_A13_kalan_isareti.py`) aynı kayan karma satırını **+ q olmadan** kullanır (Python: `ht = (d * (ht - int(t[s]) * h) + int(t[s + m])) % q`). T = 2359023141526739921, P = 31415, q = 13 için iki dil kaç karma eşitliği yazdırır; hangisi Örnek 12 ile aynıdır, neden? C'yi düzeltin.

alistirmalar/H10_A13_kalan_isareti.c · satır 25–32

```
25      for (size_t s = 0; s + m <= n; s++) {
26          if (ht == hp) {
27              esit++;
28          }
29          if (s + m < n) {
30              ht = (d * (ht - (t[s] - '0') * h) + (t[s + m] - '0')) % q;
31          }
32      }
```

Alıştırma 10.14 ★★☆ *Diller arası okuma*

`kavram/H10_K05_hash_eq_sozlesmesi.py`'yi okuyun: `Ogrenci`'de `__eq__` ve `__hash__` numaradan tanımlıdır (23–34. satırlar); 52. satırda anahtar tablodayken numarası değiştirilir. 3. bölüm ne yazdırır? `a in notlar` neden False, kayıt silindi mi, `len(notlar)` kaç? C++'ta `std::unordered_map<Ogrenci, int>` ile aynı hata yapılabilir mi (Kavram 5)? Anahtarlar için hangi kural çıkar?

11.8 Atölye 1 · Karma fonksiyonu yarışması (eşli, ≈ 25 dk)

> ALGORİTMA ATÖLYESİ · **Kendi karma fonksiyonunu tasarla: 10^5 Türkçe ad, 100 003 kova**
>
> Şablon `alistirmalar/H10_A15_karma_yarismasi.c` (ve `.py`) standart girdiden ad listesini okur ve üç fonksiyonun kova dağılımını ölçer: polinom 31, FNV-1a ve sizin `ogrenci_karma`'nız (şimdilik baytların toplamı). Ölçüt Örnek 2'dekiyle aynıdır: boş kova sayısı, en uzun kova, başarılı aramada ortalama karşılaştırma ve $\chi^2/(m-1)$ (düzgün karmada ≈ 1). $\alpha = n/m \approx 1$'de düzgün bir karmanın boş kova sayısı ≈ m·e^(−α) = 36 790 olmalıdır.

Başlangıç (bu makinede; C ve Python aynı; `python3 ../girdiler/ad_listesi_uret.py 100000 | ./H10_A15_karma_yarismasi`): polinom boş kova 36 802, en uzun 8, $\chi^2/(m-1)$ 1,001; FNV-1a boş kova 36 671, en uzun 9, $\chi^2/(m-1)$ 0,998; kendi boş kova 97 304, en uzun 136, $\chi^2/(m-1)$ 68,002.

Kurallar: kütüphane karması yok; fonksiyon belirlenimli ve C ile Python'da aynı (32 bit işaretsiz, Python'da `& 0xFFFFFFFF`). **Adımlar:** (1) toplamın kötülüğünü iki adla gösterin; (2) kendi fonksiyonunuzu yazıp ölçün; (3) m = 131072 (2^{17}) ile çalıştırın: dağılım neden bozulabilir (Bölüm 3.5)? (4) Sonuçlar tek tabloda toplanır; en iyi χ^2'nin nedenini tartışın (her bayt bütün bitleri etkilemeli).

11.9 Atölye 2 · Kelime sayacı: üç dil, iki yöntem (gruplar, ≈ 20 dk)

> ALGORİTMA ATÖLYESİ · **Kamu malı bir Türkçe metnin en sık 20 kelimesi**
>
> - Örnek 7'yi üç dilde ve Kavram 7'yi (`std::unordered_map`, `dict`) aynı girdiyle çalıştırın; en sık 20 satır birebir aynı mı (`diff`)?
> - Süreleri (stderr) karşılaştırın; bu makinedeki ortancalar için bkz. Şekil 10.20. Sıra sizde de aynı mı?
> - Dördüncü bir Ömer Seyfettin öyküsü ekleyin (Vikikaynak, kamu malı): en sık 20 değişiyor mu? Büyük harfle başlayan Türkçe sözcükler neden ayrı sayılıyor; `str.lower` ile üç dil neden ayrışırdı (Kavram 8)?

11.10 Atölye 3 · Saldırı ve savunma (iki takım, ≈ 25 dk)

> ALGORİTMA ATÖLYESİ · **Kötü karma fonksiyonunu çökert, sonra koru**
>
> - **Saldırı takımı.** h("Aa") = h("BB")'den başka çakışan iki baytlık blok bulun (bir bayt 1 artıp öbürü 31 azalırsa?) ve kendi bloklarınızla 2^{10} çakışan anahtar üretip karşılaştırmanın n(n − 1)/2 olduğunu gösterin.
> - **Savunma takımı.** Örnek 13'ün dört savunmasını (`./H10_13_karma_saldirisi 12`) deneyin; her birinin neden işe yaradığını ya da yaramadığını bir cümleyle açıklayın (Bölüm 5.6). Gizli taban değişince sonuç değişiyor mu? `kavram/H10_K09_gizli_taban.py`'nin t + s sütununu yorumlayın.
> - **Birlikte.** Deney (b)'yi kendi makinenizde çalıştırıp eğimleri karşılaştırın. Bir web sunucusu parametreleri sözlüğe koyuyorsa hangi üç önlemi alırsınız (Bölüm 9)? `PYTHONHASHSEED=0` neden yalnız testte kabul edilebilir?

12. Kendini Değerlendirme

- ☐ Sözlük ADT'sinin dört gerçekleştirimini maliyetleriyle karşılaştırabiliyorum.
- ☐ Karma yöntemlerini karşılaştırıp dağılımı χ^2 ile ölçebiliyor, çakışmayı açıklayabiliyorum.
- ☐ Ayrı zincirlemede ekle/ara/sil'i `Dugum **` ve doğru anahtar sahipliğiyle yazabiliyorum.

- ☐ Doğrusal yoklamada mezar taşının gerekliliğini değişmezle açıklayıp gerçekleyebiliyorum.
- ☐ α'nın etkisini ölçümle gösterip büyütmenin amortize O(1) olduğunu kanıtlayabiliyorum.
- ☐ Sayaç, küme, harita ve anahtar tasarımıyla $\Theta(n^2)$'yi beklenen $\Theta(n)$'e indirebiliyorum.
- ☐ Karma taşkınını, savunmaların sınırlarını ve kriptografik karma farkını açıklayabiliyorum.
- ☐ Naif eşleme ve Rabin–Karp'ı gerçekleyebiliyor, KMP'nin önek fonksiyonunu açıklayabiliyorum.

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Karma nerede yeniden karşımıza çıkacak?**

- **H11 (graflar):** BFS/DFS'nin "bu düğümü ziyaret ettim mi?" sorusu bir **ziyaret kümesidir**: düğümler 0 … n − 1 ise dizi (doğrudan adresleme), değilse karma kümesi.
- **H13 (dinamik programlama):** memoization, alt problemin girdisini anahtar, sonucunu değer yapan bir eşlemedir: durum küçük tamsayılarsa dizi, seyrekse karma tablosu (Bölüm 5.5).
- **Olasılıksal yapılar:** Bloom filtresi bir bit dizisinde "kesinlikle yok / büyük olasılıkla var" der (yanlış pozitif olabilir, yanlış negatif olmaz); MinHash küme benzerliğini küçük imzalarla tahmin eder; **tutarlı karma** sunucu değişince anahtarların yalnız küçük bir kesrini taşır.
- **Kriptografi ve dizgi algoritmaları:** SHA-2/SHA-3 Bilgi Güvenliği ve Kriptografi dersinde; Aho–Corasick ve sonek dizileri ileri algoritma derslerinde.

MERAK KUTUSU · **KMP'nin önek fonksiyonu başka neyi söyler?**

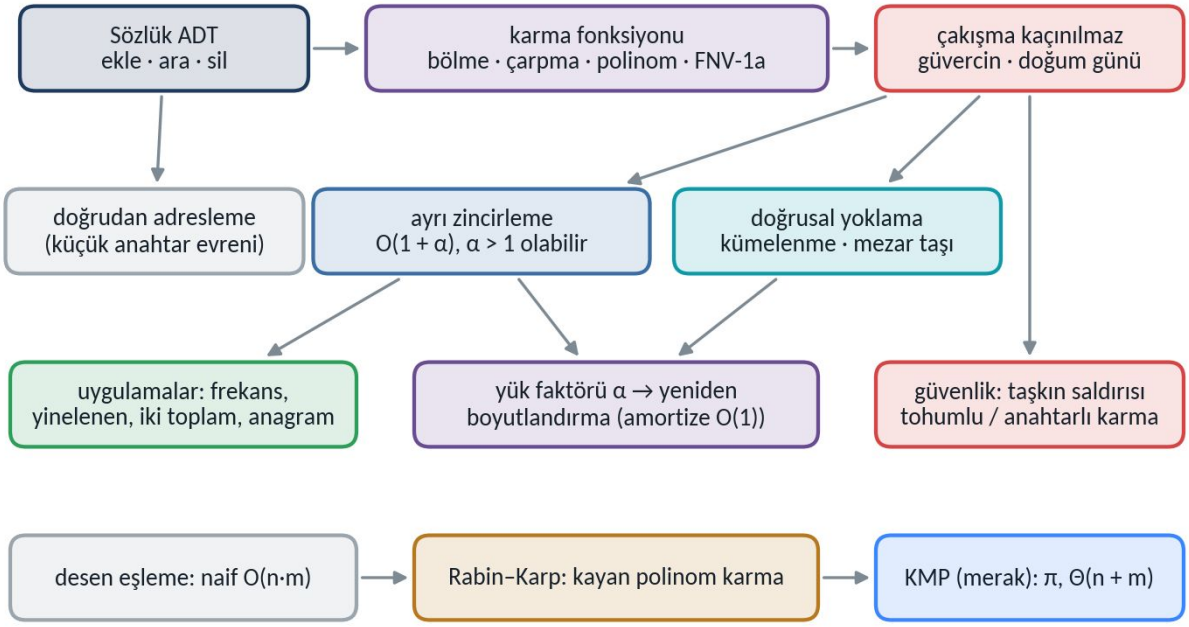
Bölüm 4.6'da π'yi desenin kendi içindeki tekrarlarını hatırlamak için kullandık: ababaca için π = [0, 0, 1, 2, 3, 0, 1]. Aynı dizi desenin **en kısa periyodunu** da verir: m − π[m]. ababaca için 7 − 1 = 6 (neredeyse hiç tekrar yok); abababab için π = [0, 0, 1, 2, 3, 4, 5, 6] ve periyot 8 − 6 = 2.

Soru: P$T dizgisinin (araya ikisinde de geçmeyen bir ayraç koyarak) önek fonksiyonu neden P'nin T'deki bütün eşleşmelerini verir?

14. Özet, Kavram Haritası ve Terim Sözlüğü

- Doğrudan adresleme $\Theta(1)$'dir ama evren kadar bellek ister; karma tablosu anahtarı m kovaya indirger. İyi karma hızlı, belirlenimli ve düzgündür (χ^2); $m = 2^p$'de maske yalnız düşük bitleri görür.
- Çakışma kaçınılmazdır ve erken gelir (m = 365'te 23 anahtarda %50): ayrı zincirleme $O(1 + \alpha)$, doğrusal yoklama + mezar taşı; eşik 3/4 ve × 2 büyütme amortize O(1). Süre bellek erişimine de bağlıdır: 10^6 kayıtta ikili arama 762 ns, karma 186 ns (v2.9 belgesi § 8.1).
- Sayaç, küme, harita, anahtar tasarımı $\Theta(n^2)$'yi beklenen $\Theta(n)$'e indirir; memoization H13'e köprüdür. Ortalama durum bir varsayımdır: gerçek savunma anahtarlı karmadır (SipHash).
- Naif eşleme en kötü $O(n \cdot m)$; Rabin–Karp kayan karma + doğrulama ile beklenen $O(n + m)$; KMP en kötü durumda da $\Theta(n + m)$.

Şekil 10.21
Hafta 10 kavram haritası



Tablo 10.16
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
karma; karma fonksiyonu; karma tablosu; kova	hashing; hash function; hash table; bucket	"karma" yalnızca hash için
sözlük (harita); küme; doğrudan adresleme	dictionary (map); set; direct addressing	ADT; anahtar = indis
çakışma; güvercin yuvası; doğum günü paradoksu	collision; pigeonhole principle; birthday paradox	
ayrı zincirleme; açık adresleme; doğrusal yoklama	separate chaining; open addressing; linear probing	zincir: H8 bağlı listesi
yoklama; kümelenme; mezar taşı	probe; (primary) clustering; tombstone	mezar taşı aramayı durdurmaz
yük faktörü; yeniden boyutlandırma; amortize maliyet	load factor; resizing (rehashing); amortized cost	α = n/m
karma taşkını; anahtarlı karma; kriptografik karma	hash flooding; keyed hash; cryptographic hash	SipHash anahtarlıdır
desen eşleme; kayan karma; sahte eşleşme; önek fonksiyonu	string matching; rolling hash; spurious hit; prefix function	

15. Kaynaklar

Aumasson, J.-P., & Bernstein, D. J. (2012). SipHash: A fast short-input PRF. In *Progress in cryptology – INDOCRYPT 2012* (pp. 489–508). Springer. https://doi.org/10.1007/978-3-642-34931-7_28

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Crosby, S. A., & Wallach, D. S. (2003). Denial of service via algorithmic complexity attacks. In *Proceedings of the 12th USENIX Security Symposium* (pp. 29–44). USENIX Association.

Heimes, C. (2013). *PEP 456 – Secure and interchangeable hash algorithm*. Python Software Foundation. https://peps.python.org/pep-0456/

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Karp, R. M., & Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, *31*(2), 249–260. https://doi.org/10.1147/rd.312.0249

Klink, A., & Wälde, J. (2011, December 28). *Effective denial of service attacks against web application platforms* [Conference presentation]. 28th Chaos Communication Congress (28C3). https://media.ccc.de/v/28c3-4680-en-effective_dos_attacks_against_web_application_platforms

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Knuth, D. E., Morris, J. H., Jr., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, *6*(2), 323–350. https://doi.org/10.1137/0206024

Noll, L. C. (t.y.). *FNV hash*. Erişim tarihi 13 Ekim 2026, http://www.isthe.com/chongo/tech/comp/fnv/index.html

oCERT. (2011, December 28). *oCERT-2011-003: Multiple implementations denial-of-service via hash algorithm collision*. http://www.ocert.org/advisories/ocert-2011-003.html

Parsons, D., & Haden, P. (2006). Parson's programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Polat, N. H. (2007). Ömer Seyfeddin. In *TDV İslâm ansiklopedisi* (Vol. 34, pp. 80–82). TDV İslâm Araştırmaları Merkezi. https://islamansiklopedisi.org.tr/omer-seyfeddin

Python Software Foundation. (t.y.-a). *Objects/dictobject.c* (CPython 3.11) [Kaynak kod]. GitHub. Erişim tarihi 13 Ekim 2026, https://github.com/python/cpython/blob/3.11/Objects/dictobject.c

Python Software Foundation. (t.y.-b). *What's new in Python 3.11*. Erişim tarihi 13 Ekim 2026, https://docs.python.org/3/whatsnew/3.11.html

Python Software Foundation. (t.y.-c). *What's new in Python 3.3*. Erişim tarihi 13 Ekim 2026, https://docs.python.org/3/whatsnew/3.3.html

Python Software Foundation. (t.y.-d). *What's new in Python 3.7*. Erişim tarihi 13 Ekim 2026, https://docs.python.org/3/whatsnew/3.7.html

Seyfettin, Ö. (2022). *Diyet*. Vikikaynak. https://tr.wikisource.org/w/index.php?title=Diyet&oldid=151645 (Özgün eser 1918'de yayımlandı)

Seyfettin, Ö. (2023). *Ferman*. Vikikaynak. https://tr.wikisource.org/w/index.php?title=Ferman&oldid=163811 (Özgün eser 1917'de yayımlandı)

Seyfettin, Ö. (2024). *Forsa*. Vikikaynak. https://tr.wikisource.org/w/index.php?title=Forsa&oldid=167612 (Özgün eser 1919'da yayımlandı)

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on dört basamağının C++ ve Python sürümlerinden çekirdek fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Bölüm 4'te üç dili gösterilen fonksiyonlar burada tekrarlanmaz; aynı dosyaların öteki çekirdek fonksiyonları verilir. Bütün örneklerin çıktısı üç dilde aynıdır (=).

Tablo 10.17
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H10_03_ayri_zincirleme)

Dil	Derleme / çalıştırma	Not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c [ortak_rastgele.c] -o X -lm · ./X`	ortak_rastgele.c yalnız #include eden örneklerde
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Girdi	`./H10_07_kelime_frekansi < ../girdiler/H10_07_kelime_frekansi.txt (Örnek 2 ve 9 da girdi dosyası okur)`	girdi dosyaları kodlar/girdiler/ altında

A.1 Örnek 1 · Doğrudan adresleme

C++ cpp/H10_01_dogrudan_adresleme.cpp · satır 27–43

```cpp
static void ekle(int no, const std::string& ad)
{
    erisim_sayisi++;
    tablo[no] = ad;              // anahtar = indis: arama yok
}

static const std::string& ara(int no)
{
    erisim_sayisi++;
    return tablo[no];
}

static void sil(int no)
{
    erisim_sayisi++;
    tablo[no].clear();
}
```

Python python/H10_01_dogrudan_adresleme.py · satır 24–39

```python
def ekle(no, ad):
    global erisim_sayisi
    erisim_sayisi += 1
    tablo[no] = ad               # anahtar = indis: arama yok

```

```
29
30 def ara(no):
31     global erisim_sayisi
32     erisim_sayisi += 1
33     return tablo[no]
34
35
36 def sil(no):
37     global erisim_sayisi
38     erisim_sayisi += 1
39     tablo[no] = None
```

DİL PENCERESİ · Farklılaşan satırlar

Boş hücre C'de `NULL` işaretçisidir (hücre, dizgi değişmezinin adresini tutar); C++'ta boş `std::string` (hücre adın kopyasını tutar), Python'da `None`. Bellek hesabında hücre boyutu sabit 8 bayt (x86-64 LP64'te bir işaretçi) olarak verilir: `sizeof` çıktıya yazılmaz, çünkü Python'da karşılığı yoktur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 23, 125; Python 17, 98.)

A.2 Örnek 2 · Dizgi karma fonksiyonları

C++ **cpp/H10_02_karma_fonksiyonlari.cpp · satır 35–53**

```
35 static std::uint32_t karma_polinom(const std::string& s)
36 {
37     std::uint32_t h = 0;
38     for (unsigned char b : s) {
39         h = 31u * h + b;
40     }
41     return h;
42 }
43
44 // FNV-1a 32 bit (Fowler, Noll, Vo): önce XOR, sonra FNV asalıyla çarp
45 static std::uint32_t karma_fnv1a(const std::string& s)
46 {
47     std::uint32_t h = 2166136261u;      // FNV başlangıç değeri
48     for (unsigned char b : s) {
49         h ^= b;
50         h *= 16777619u;                 // FNV asalı (2^24 + 2^8 + 0x93)
51     }
52     return h;
53 }
```

Python **python/H10_02_karma_fonksiyonlari.py · satır 27–41**

```
27 def karma_polinom(s):
28     """Polinom (Horner, taban 31): h = 31·h + bayt; & MASKE32 ile sarar"""
29     h = 0
30     for b in s:
31         h = (31 * h + b) & MASKE32
32     return h
33
34
```

```python
def karma_fnv1a(s):
    """FNV-1a 32 bit (Fowler, Noll, Vo): önce XOR, sonra FNV asalıyla"""
    h = 2166136261                      # FNV başlangıç değeri
    for b in s:
        h ^= b
        h = (h * 16777619) & MASKE32    # FNV asalı (2^24 + 2^8 + 0x93)
    return h
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta `uint32_t` aritmetiği mod 2^32 sarar (işaretsiz taşma tanımlıdır); Python tamsayısı büyüdüğü için her adımda `& 0xFFFFFFFF` uygulanır. `char` x86-64'te işaretlidir: C'de baytlar `(unsigned char)`'a çevrilir, C++'ta `for (unsigned char b : s)`; Python'da girdi `sys.stdin.buffer` ile bayt olarak okunur ve `bytes` öğeleri zaten 0–255 arasıdır. `hash()`/`std::hash` çıktıda kullanılmaz. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 28, 154; Python 16, 110.)

A.3 Örnek 3 · Ayrı zincirleme

C++ `cpp/H10_03_ayri_zincirleme.cpp · satır 99–115`

```cpp
static bool tablo_sil(Tablo& t, const std::string& anahtar)
{
    Dugum **bag = &t.kova[kova_no(t, anahtar)];
    while (*bag != nullptr) {
        karsilastirma_sayisi++;
        if ((*bag)->anahtar == anahtar) {
            Dugum *silinen = *bag;
            *bag = silinen->sonraki;
            delete silinen;
            birakilan_dugum++;
            t.n--;
            return true;
        }
        bag = &(*bag)->sonraki;
    }
    return false;
}
```

Python `python/H10_03_ayri_zincirleme.py · satır 79–97`

```python
def tablo_sil(t, anahtar):
    """Python'da Dugum ** yok: önceki düğümü izleyerek bağlantıyı atla"""
    global karsilastirma_sayisi, birakilan_dugum
    j = kova_no(t, anahtar)
    onceki = None
    d = t.kova[j]
    while d is not None:
        karsilastirma_sayisi += 1
        if d.anahtar == anahtar:
            if onceki is None:
                t.kova[j] = d.sonraki
            else:
                onceki.sonraki = d.sonraki
```

```
            birakilan_dugum += 1          # son başvuru gidince toplanır
            t.n -= 1
            return True
        onceki = d
        d = d.sonraki
    return False
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de anahtar `dizgi_kopyala` (malloc + memcpy; `strdup` C17'de yok) ile kopyalanır ve `tablo_yok_et`'te serbest bırakılır; C++'ta `std::string` üyesi kopyayı kendisi yapar, düğümler `new`/`delete` ile ve yıkıcıda döngüyle silinir (RAII); Python'da `bytes(tampon)` değişmez bir kopya üretir, düğümler başvuru kalmayınca toplanır (sayaç elle tutulur). Silme C/C++'ta `Dugum **` ile; Python'da işaretçiye işaretçi olmadığından önceki düğüm izlenir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 20, 39, 79; Python 13, 67.)

A.4 Örnek 4 · Doğrusal yoklama ve mezar taşı

C++ `cpp/H10_04_dogrusal_yoklama.cpp · satır 59–86`

```
static long bul(const Tablo& t, const std::string& anahtar, long& yer)
{
    std::size_t m = t.hucre.size();
    std::size_t i = ev(t, anahtar);
    long ilk_mezar = -1;
    yol.clear();
    for (std::size_t adim = 0; adim < m; adim++) {
        yoklama_sayisi++;
        if (yol.size() < 64) {
            yol.push_back(i);
        }
        const Hucre& h = t.hucre[i];
        if (h.durum == BOS) {
            yer = ilk_mezar >= 0 ? ilk_mezar : static_cast<long>(i);
            return -1;
        }
        if (h.durum == MEZAR) {
            if (ilk_mezar < 0) {
                ilk_mezar = static_cast<long>(i);
            }
        } else if (h.anahtar == anahtar) {
            return static_cast<long>(i);
        }
        i = (i + 1) % m;                    // doğrusal yoklama
    }
    yer = ilk_mezar;                        // BOŞ hücre hiç yok
    return -1;
}
```

Python `python/H10_04_dogrusal_yoklama.py · satır 51–71`

```
def bul(t, anahtar):
    """(indis ya da -1, yer): yer = yoldaki İLK mezar taşı, yoksa durulan
    BOŞ hücre; tablo doluysa -1. Mezar taşı aramayı DURDURMAZ."""
```

```
    global yoklama_sayisi
    i = ev(t, anahtar)
    ilk_mezar = -1
    yol.clear()
    for _ in range(t.m):
        yoklama_sayisi += 1
        if len(yol) < 64:
            yol.append(i)
        h = t.hucre[i]
        if h.durum == BOS:
            return -1, (ilk_mezar if ilk_mezar >= 0 else i)
        if h.durum == MEZAR:
            if ilk_mezar < 0:
                ilk_mezar = i
        elif h.anahtar == anahtar:
            return i, -1
        i = (i + 1) % t.m                    # doğrusal yoklama
    return -1, ilk_mezar                     # BOŞ hücre hiç yok
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> `bul` iki sonuç döndürür: C'de çıktı parametresi (`long *yer`), C++'ta başvuru (`long& yer`), Python'da demet `(i, yer)`. Rastgele anahtar metni C'de `snprintf`, C++'ta `std::to_string`, Python'da f-dizgisi + `encode` ile üretilir (üçü aynı ondalık yazımı verir). Yeniden kurmada C anahtar işaretçilerini, C++ `std::move` ile dizgileri, Python `Hucre` nesnelerini taşır — kopya yoktur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 32, 150, 219; Python 86, 171.)

A.5 Örnek 5 · Yük faktörü deneyi

C++ **cpp/H10_05_yuk_faktoru.cpp · satır 52–68**

```
static bool dy_ara(const std::vector<std::uint32_t>& anahtar,
                   const std::vector<unsigned char>& dolu, int p,
                   std::uint32_t k)
{
    std::uint32_t maske = (1u << p) - 1u;
    std::uint32_t i = karma_carpma(k, p);
    for (;;) {
        yoklama_sayisi++;                    // bakılan her hücre
        if (!dolu[i]) {
            return false;
        }
        if (anahtar[i] == k) {
            return true;
        }
        i = (i + 1u) & maske;
    }
}
```

Python **python/H10_05_yuk_faktoru.py · satır 46–56**

```
def dy_ara(anahtar, dolu, p, k):
    global yoklama_sayisi
```

```
    maske = (1 << p) - 1
    i = karma_carpma(k, p)
    while True:
        yoklama_sayisi += 1                 # bakılan her hücre
        if not dolu[i]:
            return False
        if anahtar[i] == k:
            return True
        i = (i + 1) & maske
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Çarpma yönteminde C/C++'ta `k * 2654435769u` 32 bitte kendiliğinden sarar; Python'da `& 0xFFFFFFFF` gerekir, ardından üst p bit sağa kaydırmayla alınır. Zincir düğümleri C'de `malloc`/`free`, C++'ta `new`/`delete` + RAII sarmalayıcı (`Kovalar`), Python'da nesne + başvuru bırakma. Komut satırı `strtol` / `std::stol` / `int()` ile denetlenir; hatalı kullanımda üç dilde çıkış kodu 2. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 71, 76, 113, 155; Python 30, 78, 136.)

A.6 Örnek 6 · Yeniden boyutlandırma

C++ **cpp/H10_06_yeniden_boyutlandirma.cpp · satır 43–62**

```
43 static bool yerlestir(Tablo& t, std::uint32_t k)
44 {
45     std::uint32_t maske = (std::uint32_t{1} << t.p) - 1u;
46     std::uint32_t i = karma_carpma(k, t.p);
47     for (;;) {
48         yoklama_sayisi++;
49         if (!t.dolu[i]) {
50             break;
51         }
52         if (t.anahtar[i] == k) {
53             return false;
54         }
55         i = (i + 1u) & maske;
56     }
57     t.dolu[i] = 1;
58     t.anahtar[i] = k;
59     t.n++;
60     yazma_sayisi++;
61     return true;
62 }
```

Python **python/H10_06_yeniden_boyutlandirma.py · satır 37–53**

```
37 def yerlestir(t, k):
38     """Yer bulup yazar (yük denetimi yok); True: yeni, False: zaten var"""
39     global yoklama_sayisi, yazma_sayisi
40     maske = (1 << t.p) - 1
41     i = karma_carpma(k, t.p)
42     while True:
43         yoklama_sayisi += 1
44         if not t.dolu[i]:
```

```
            break
        if t.anahtar[i] == k:
            return False
        i = (i + 1) & maske
    t.dolu[i] = 1
    t.anahtar[i] = k
    t.n += 1
    yazma_sayisi += 1
    return True
```

DİL PENCERESİ · **Farklılaşan satırlar**

C'de büyütme yeni dizileri ayırır, eskilerini `free` eder ve yapıyı `*t = yeni` ile değiştirir; C++'ta `t = std::move(yeni)` vektörleri taşır (eski bellek kendiliğinden serbest kalır); Python'da yeni listeler bağlanır, eskileri çöp toplayıcıya kalır. Python listesinin kendi büyümesine güvenilmez: kapasite `[0] * (1 << p)` ile baştan ayrılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 28, 81; Python 21, 69.)

A.7 Örnek 7 · Kelime frekansı

C++ **cpp/H10_07_kelime_frekansi.cpp · satır 81–95**

```
81 static void say(Tablo& t, const std::string& sozcuk)
82 {
83     std::size_t j = karma_fnv1a(sozcuk) & (t.kova.size() - 1);
84     for (Dugum *d = t.kova[j]; d != nullptr; d = d->sonraki) {
85         if (d->anahtar == sozcuk) {
86             d->deger++;
87             return;
88         }
89     }
90     t.kova[j] = new Dugum{sozcuk, 1, t.kova[j]};
91     t.n++;
92     if (t.n > t.kova.size()) {
93         buyut(t);
94     }
95 }
```

Python **python/H10_07_kelime_frekansi.py · satır 64–76**

```
64 def say(t, sozcuk):
65     """Sözcüğün sıklığını 1 artırır (yoksa 1 ile ekler)"""
66     j = karma_fnv1a(sozcuk) & (t.m - 1)
67     d = t.kova[j]
68     while d is not None:
69         if d.anahtar == sozcuk:
70             d.deger += 1
71             return
72         d = d.sonraki
73     t.kova[j] = Dugum(bytes(sozcuk), 1, t.kova[j])
74     t.n += 1
75     if t.n > t.m:
76         buyut(t)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Sözcükler bayt düzeyinde ayrılır ve ASCII A–Z elle +32 ile küçültülür; `tolower` (yerel ayara bağlı) ve `str.lower` (Unicode kuralları: İ → i) kullanılmaz, bu yüzden Türkçe büyük harfler (Ç Ğ İ Ö Ş Ü) olduğu gibi kalır. En sık 20'yi yazmak için kütüphane sıralaması (`qsort` / `std::sort` / `list.sort`) kullanılır: bu algoritma değil, çıktı biçimidir. Eşitlikte bayt sırası: `strcmp` ve `std::string` karşılaştırması `unsigned char` olarak, Python `bytes` bayt değeriyle karşılaştırır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 109, 115, 145; Python 86, 94, 111.)

A.8 Örnek 8 · Yinelenen tespiti ve iki toplam

C++ **cpp/H10_08_yinelenen_iki_toplam.cpp · satır 164–187**

```cpp
static bool iki_toplam_karma(const std::vector<std::uint32_t>& a,
                             std::uint32_t T, std::size_t& bi,
                             std::size_t& bj)
{
    Harita h(a.size());
    for (std::size_t j = 0; j < a.size(); j++) {
        bool var;
        if (a[j] <= T) {
            std::size_t yer = harita_yer(h, T - a[j], var);
            if (var) {
                bi = static_cast<std::size_t>(h.deger[yer]);
                bj = j;
                return true;
            }
        }
        std::size_t yer = harita_yer(h, a[j], var);
        if (!var) {                          // yalnız ilk konum tutulur
            h.dolu[yer] = 1;
            h.anahtar[yer] = a[j];
            h.deger[yer] = static_cast<int>(j);
        }
    }
    return false;
}
```

Python **python/H10_08_yinelenen_iki_toplam.py · satır 126–139**

```python
def iki_toplam_karma(a, T):
    """Karma: soldan sağa; T - a[j] daha önce görüldüyse ilk görüldüğü i"""
    h = Harita(len(a))
    for j in range(len(a)):
        if a[j] <= T:
            yer, var = harita_yer(h, T - a[j])
            if var:
                return h.deger[yer], j
        yer, var = harita_yer(h, a[j])
        if not var:                          # yalnız ilk konum tutulur
            h.dolu[yer] = 1
            h.anahtar[yer] = a[j]
            h.deger[yer] = j
    return None
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Yardımcı iki sonucu C'de çıktı parametresiyle (`int *var`), C++'ta `bool&` ile, Python'da demetle döndürür. `a[i] + a[j]` C/C++'ta `uint32_t`'de taşabileceği için 64 bite genişletilir; Python tamsayısı taşmaz. 2400000000 + i öğrenci numarası `int`'e sığmaz ($2^{31} - 1 = 2147483647$): C/C++'ta `uint32_t` kullanılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 24, 137, 153; Python 37, 121, 179.)

A.9 Örnek 9 · Anagram gruplama

C++ `cpp/H10_09_anagram_gruplama.cpp · satır 60–72`

```cpp
static std::string anahtar_yap(const std::string& sozcuk)
{
    int sayac[256] = {0};
    for (unsigned char b : sozcuk) {
        sayac[b]++;
    }
    std::string anahtar;
    for (int b = 1; b < 256; b++) {
        anahtar.append(static_cast<std::size_t>(sayac[b]),
                       static_cast<char>(b));
    }
    return anahtar;
}
```

Python `python/H10_09_anagram_gruplama.py · satır 36–44`

```python
def anahtar_yap(sozcuk):
    """Anahtar: baytların sayma sıralaması (256 sayaç; 0(L + 256))"""
    sayac = [0] * 256
    for b in sozcuk:
        sayac[b] += 1
    anahtar = bytearray()
    for b in range(1, 256):
        anahtar += bytes([b]) * sayac[b]
    return bytes(anahtar)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Anahtar baytların sayma sıralamasıdır (256 sayaç): C'de `char` dizisi, C++'ta `std::string::append(sayı, karakter)`, Python'da `bytes` çarpımı. Python'da anahtar `bytes`'tır; `list` sözlük ya da küme anahtarı olamaz (`TypeError: unhashable type` — Hata Avcısı 6). Grup üyeleri C'de bağlı liste, C++'ta `std::vector`, Python'da listedir; çıktı için `qsort` / `std::sort` / `sorted` (çıktı biçimi). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 25, 139; Python 20, 93.)

A.10 Örnek 10 · Doğum günü paradoksu

C++ `cpp/H10_10_dogum_gunu.cpp · satır 37–47`

```cpp
static bool deneme_yap(int n, std::vector<int>& gorulen, int damga)
```

```
{
    for (int k = 0; k < n; k++) {
        int g = rastgele_aralik(0, GUN - 1);
        if (gorulen[g] == damga) {
            return true;
        }
        gorulen[g] = damga;
    }
    return false;
}
```

Python `python/H10_10_dogum_gunu.py · satır 28–36`

```
def deneme_yap(n, gorulen, damga):
    """Bir deneme: n doğum günü çek; ilk tekrar görülürse True. gorulen =
    doğrudan adresleme; damga = deneme numarası (listeyi sıfırlamadan)"""
    for _ in range(n):
        g = rastgele_aralik(0, GUN - 1)
        if gorulen[g] == damga:
            return True
        gorulen[g] = damga
    return False
```

DİL PENCERESİ · **Farklılaşan satırlar**

Kayan nokta işlemleri üç dilde aynı sırayla yapılır: q = q · ((m − k) / m); C/C++'ta `(double)` dönüşümü açıktır, Python'da `/` her zaman kayan noktalı bölmedir. İşaretli biçim `%+7.4f` / `std::showpos` / `{:+7.4f}`. Görülen günler doğrudan adresleme dizisinde 'damga' (deneme numarası) ile işaretlenir, dizi her denemede sıfırlanmaz. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 73, 105; Python 23, 80.)

A.11 Örnek 11 · Naif desen eşleme

C++ `cpp/H10_11_naif_eslesme.cpp · satır 25–48`

```
static std::vector<std::size_t> naif_ara(const std::string& t,
                                         const std::string& p)
{
    std::vector<std::size_t> konum;
    std::size_t n = t.size();
    std::size_t m = p.size();
    if (m == 0 || m > n) {
        return konum;
    }
    for (std::size_t s = 0; s + m <= n; s++) {   // s: kayma (shift)
        std::size_t j = 0;
        while (j < m) {
            karsilastirma_sayisi++;
            if (t[s + j] != p[j]) {
                break;                       // ilk uyuşmazlıkta bırak
            }
            j++;
        }
        if (j == m) {
```

```
            konum.push_back(s);
        }
    }
    return konum;
}
```

Python `python/H10_11_naif_eslesme.py · satır 17–34`

```python
def naif_ara(t, p):
    """t içinde p'nin tüm konumları (t, p: bytes)"""
    global karsilastirma_sayisi
    konum = []
    n = len(t)
    m = len(p)
    if m == 0 or m > n:
        return konum
    for s in range(n - m + 1):              # s: kayma (shift)
        j = 0
        while j < m:
            karsilastirma_sayisi += 1
            if t[s + j] != p[j]:
                break                       # ilk uyuşmazlıkta bırak
            j += 1
        if j == m:
            konum.append(s)
    return konum
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kâhin C'de `strstr`, C++'ta `std::string::find`, Python'da `bytes.find` döngüsüdür; çakışan eşleşmeler için sonraki arama q + 1'den başlar. Konumlar bayt indisidir: UTF-8'de "ı" ve "ç" ikişer bayttır; karakter konumu devam baytları (10xxxxxx) atlanarak sayılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 55, 103; Python 40, 97.)

A.12 Örnek 12 · Rabin–Karp

C++ `cpp/H10_12_rabin_karp.cpp · satır 56–74`

```cpp
static Sonuc rk_ara(const std::string& t, const std::string& p, long long d,
                    long long q)
{
    Sonuc r;
    std::size_t n = t.size();
    std::size_t m = p.size();
    if (m == 0 || m > n) {
        return r;
    }
    long long h = 1;                        // d^(m-1) mod q
    for (std::size_t i = 1; i < m; i++) {
        h = h * d % q;
    }
    long long hp = 0;
    long long ht = 0;
```

```
    for (std::size_t i = 0; i < m; i++) {    // Horner
        hp = (d * hp + bayt(p, i)) % q;
        ht = (d * ht + bayt(t, i)) % q;
    }
```

Python `python/H10_12_rabin_karp.py · satır 44–58`

```python
def rk_ara(t, p, d, q):
    """Asal modüllü Rabin–Karp (t, p: bytes; Python tamsayısı taşmaz)"""
    r = Sonuc()
    n = len(t)
    m = len(p)
    if m == 0 or m > n:
        return r
    h = 1                                    # d^(m-1) mod q
    for _ in range(1, m):
        h = h * d % q
    hp = 0
    ht = 0
    for i in range(m):                       # Horner
        hp = (d * hp + p[i]) % q
        ht = (d * ht + t[i]) % q
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta `%` bölünenin işaretini alır (−12 % 13 = −12), Python'da bölenin (−12 % 13 = 1): kayan karmada `T[s]·h % q` çıkarılmadan önce + q eklenir, böylece ara değer üç dilde de negatif olmaz. mod 2^32 sürümünde C/C++ `uint32_t` çıkarması tanımlı biçimde sarar; Python'da negatif ara değer `& 0xFFFFFFFF` ile aynı sonuca iner. Programın 2. bölümündeki 'C/C++: x % q' sütunu Python'da `c_kalan` ile açıkça hesaplanır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 50, 115; Python 65, 93, 123.)

A.13 Örnek 13 · Karma saldırısı

C++ `cpp/H10_13_karma_saldirisi.cpp · satır 63–70`

```
static std::uint32_t karma_polinom(const std::string& s, std::uint32_t h,
                                   std::uint32_t taban)
{
    for (unsigned char b : s) {
        h = taban * h + b;
    }
    return h;
}
```

C++ **cpp/H10_13_karma_saldirisi.cpp · satır 150–157**

```cpp
static std::string carpisan_anahtar(std::size_t i, int k)
{
    std::string s;
    for (int b = 0; b < k; b++) {
        s += (i >> (k - 1 - b)) & 1u ? "BB" : "Aa";
    }
    return s;
}
```

Python **python/H10_13_karma_saldirisi.py · satır 47–51**

```python
def karma_polinom(s, h, taban):
    """Polinom 31 (Java String.hashCode'un formülü; UTF-8 baytlarıyla)"""
    for b in s:
        h = (taban * h + b) & MASKE32
    return h
```

Python **python/H10_13_karma_saldirisi.py · satır 119–122**

```python
def carpisan_anahtar(i, k):
    """i'nin k bitinden anahtar: bit 0 → "Aa", 1 → "BB" (anlamlı bit önce)"""
    return b"".join(b"BB" if (i >> (k - 1 - b)) & 1 else b"Aa"
                    for b in range(k))
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Aa/BB çakışması tam sayı aritmetiğinde de vardır (65·31 + 97 = 66·31 + 66); 32 bitlik karma onu korur: C/C++'ta `uint32_t` çarpımı mod 2^32 sarar, Python'da her adımda `& 0xFFFFFFFF` uygulanır ve üç dil aynı karmaları verir. Anahtarlar C'de sabit boyutlu `char` dizilerine, C++'ta `std::string`'e, Python'da `bytes`'a yazılır. Süreler yalnızca stderr'e yazılır: karşılaştırma sayısı dilden bağımsızdır, süre değildir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 131, 302; Python 169.)

A.14 Örnek 14 · KMP önek fonksiyonu

C++ **cpp/H10_14_kmp.cpp · satır 36–54**

```cpp
static std::vector<std::size_t> onek_fonksiyonu(const std::string& p)
{
    std::vector<std::size_t> pi(p.size(), 0);
    std::size_t k = 0;
    for (std::size_t q = 1; q < p.size(); q++) {
        for (;;) {
            if (esit_mi(p[k], p[q])) {
                k++;
                break;
            }
            if (k == 0) {
                break;
            }
            k = pi[k - 1];              // daha kısa sınıra düş
        }
```

```
        pi[q] = k;
    }
    return pi;
}
```

Python **python/H10_14_kmp.py · satır 27–41**

```python
def onek_fonksiyonu(p):
    """pi[q] = p[0..q]'nun hem öneki hem soneki olan en uzun (kendisinden
    kısa) parçanın uzunluğu. 0(m) karşılaştırma (en çok 2m)"""
    pi = [0] * len(p)
    k = 0
    for q in range(1, len(p)):
        while True:
            if esit_mi(p[k], p[q]):
                k += 1
                break
            if k == 0:
                break
            k = pi[k - 1]               # daha kısa sınıra düş
        pi[q] = k
    return pi
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Döngü C ve C++'ta `for (;;) { … break; }`, Python'da `while True:` … `break` biçimindedir. Konum listeleri C'de dizi + sayaç, C++'ta `std::vector`, Python'da liste; C++ ve Python'da listeler `==` ile doğrudan karşılaştırılır. Kâhin `strstr` / `std::string::find` / `bytes.find`. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 183; Python 52, 150.)